

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

THIẾT KẾ GIAO TIẾP GIỮA CÁC DỊCH VỤ

Các loại giao tiếp trong microservices: giao tiếp đồng bộ (synchronous) và bất đồng bộ (asynchronous) giữa các dịch vụ.

Các giao thức giao tiếp microservices phổ biến: RESTful API, GraphQL API, gRPC cho hiệu năng cao giữa các dịch vụ nội bộ, và WebSocket API cho giao tiếp hai chiều thời gian thực.

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Microservices Communications – The Basics

Microservices Communication Types - Synchronous or Asynchronous Communication

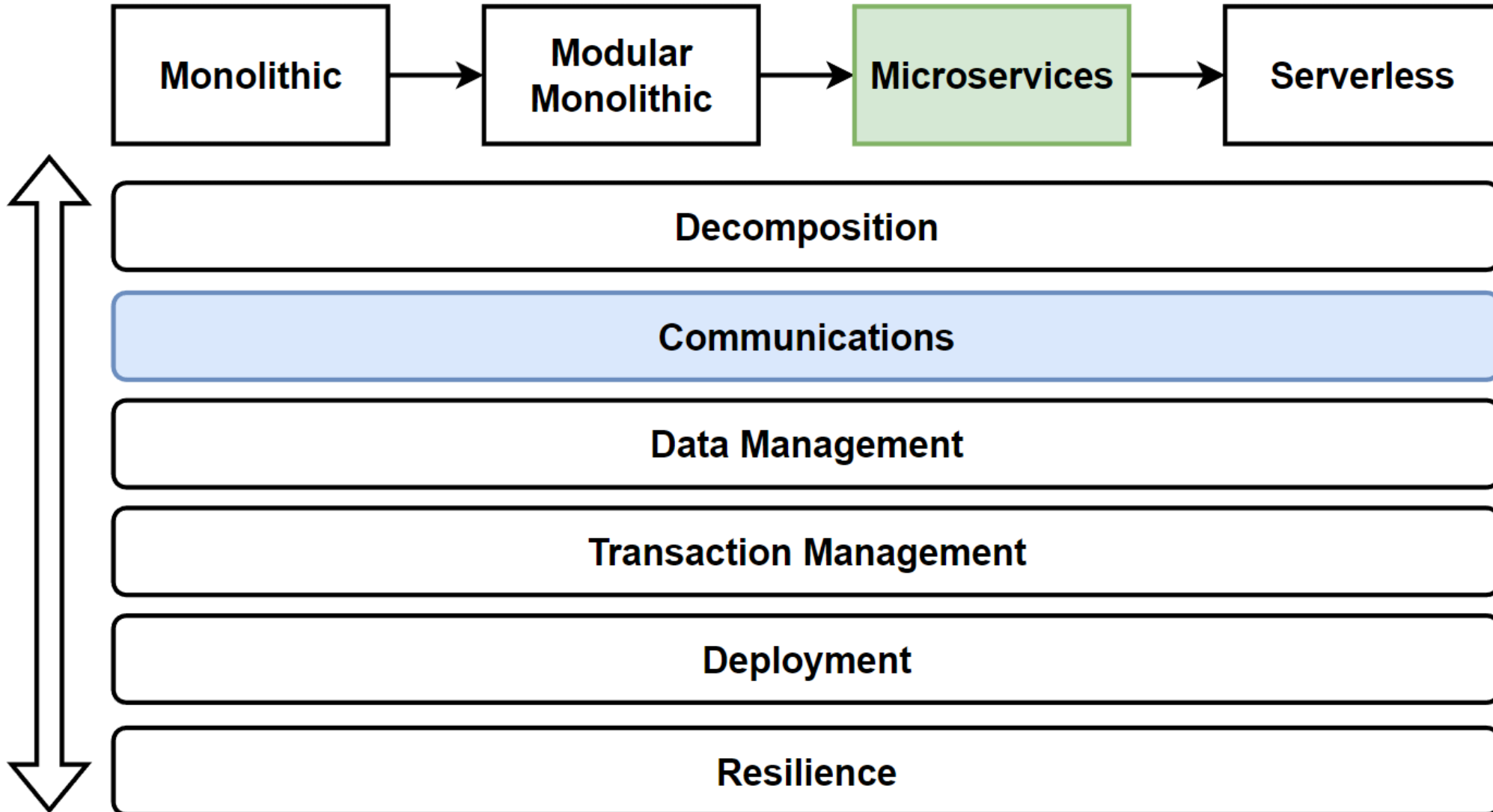
RESTful API design for Microservices

GraphQL API flexible structured relational data for Microservices

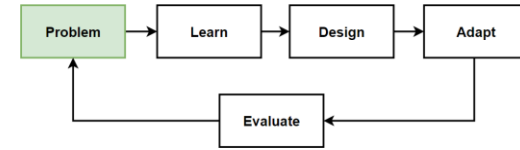
gRPC high performance communication between internal Microservices

WebSocket API Real-time two-way communications

Architecture Design – Vertical Considerations



Problem: Direct Client-to-Service Communication



Problems

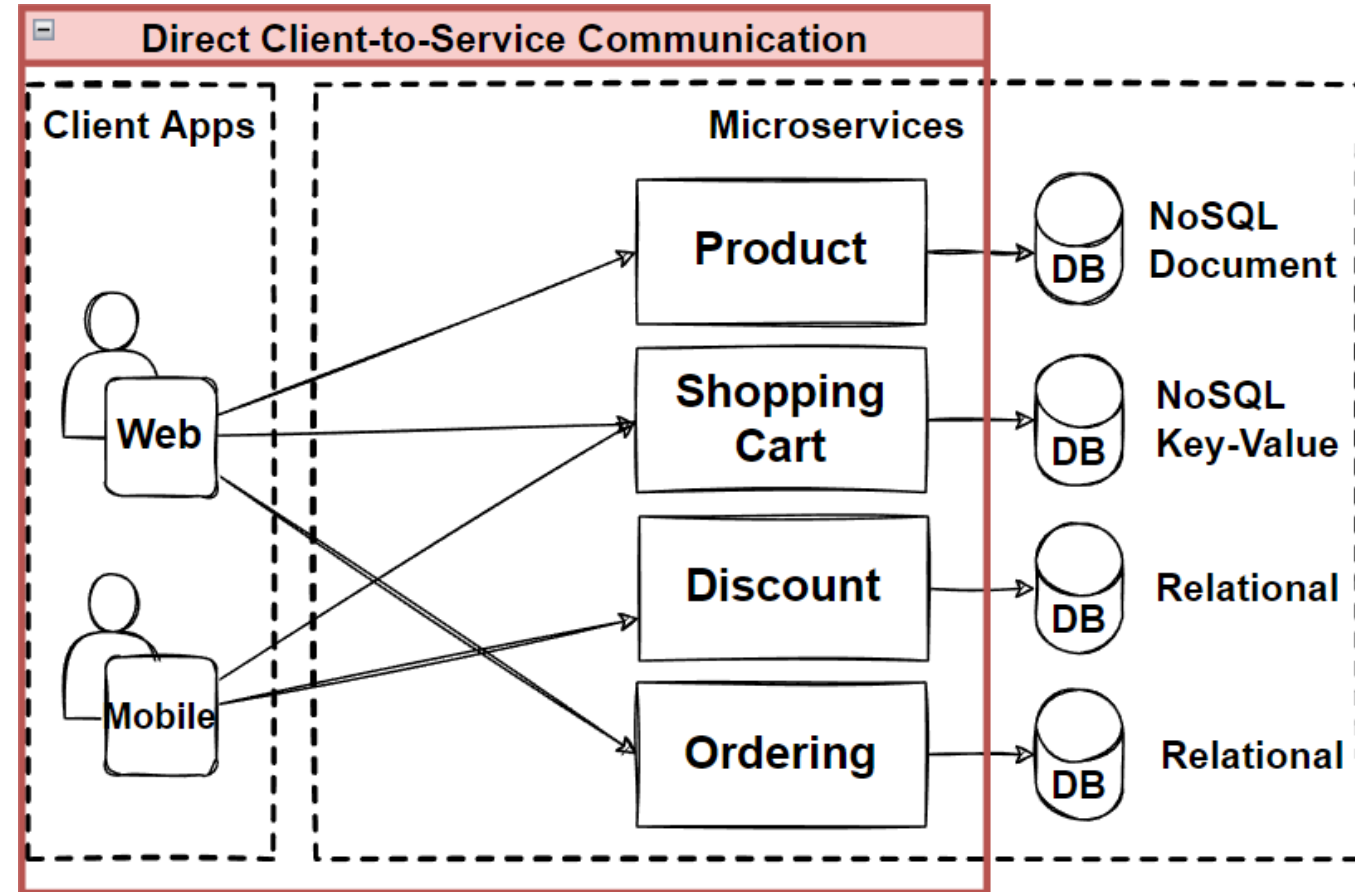
- Direct Client-to-Service Communication
- Cause to chatty calls from client to service
- Hard to manage invocations from client app.

Solutions

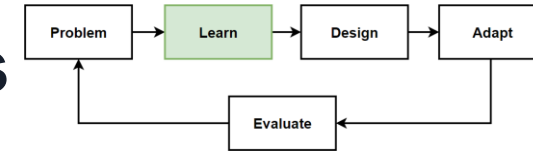
- Well-defined API Design
- Microservices Communication Patterns

Steps

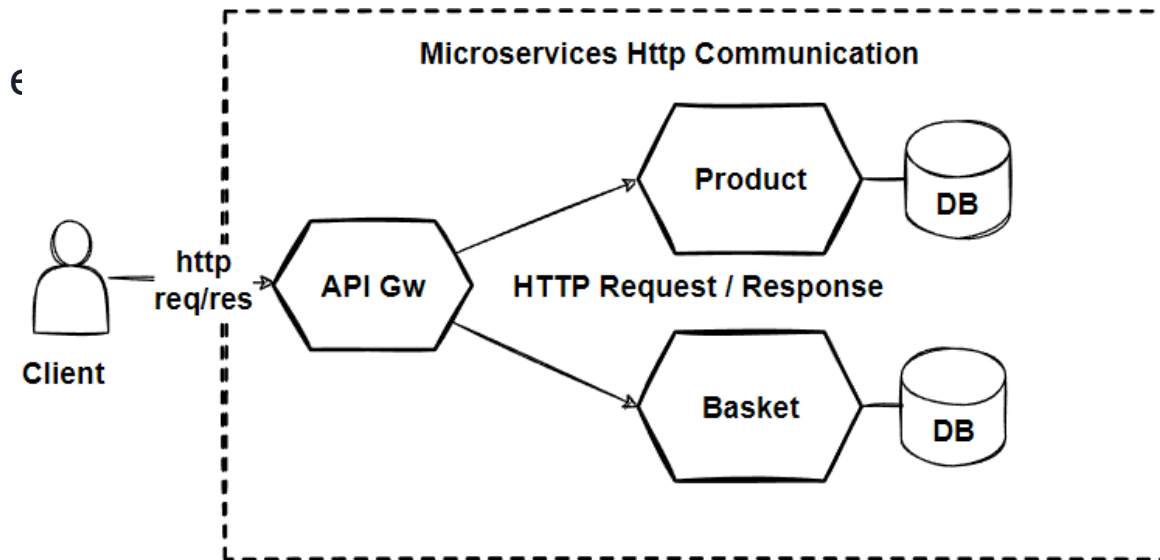
- Microservice Communications and Types
- Well-defined RESTful API Design for services
- Microservices Communication Patterns (API Gateway, BFF, Publish/Subscribe..)



Communications Between Monolithic to Microservices

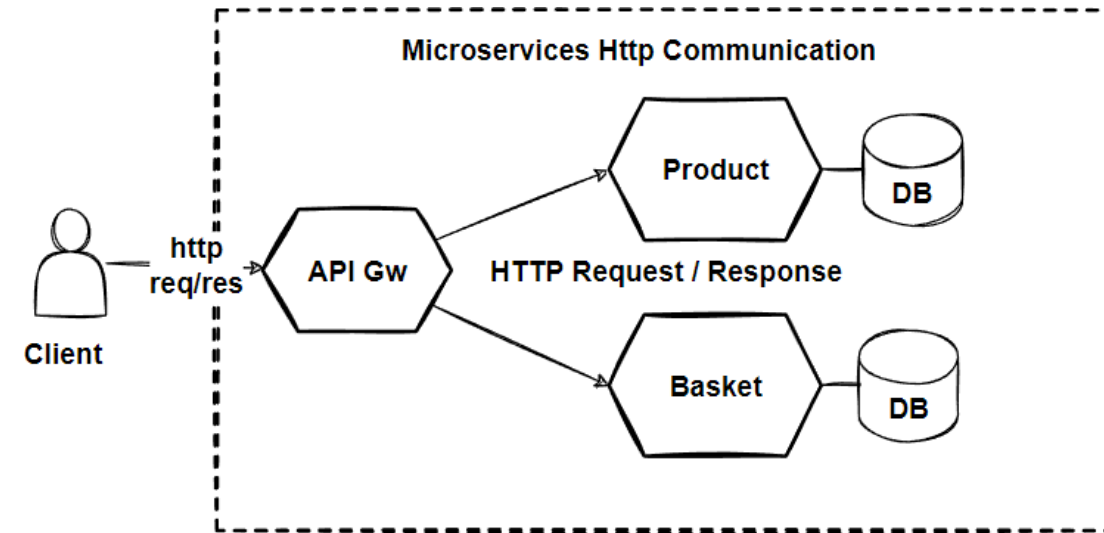
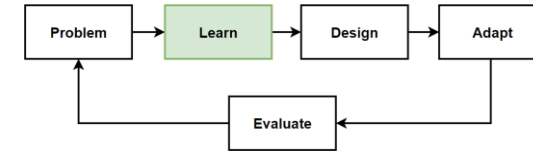


- The communication in **Monolithic** applications are **inter-process** communication.
- Working on **single process** that invoke one to another by using **method calls**. Create class and call the method inside of target module. All running the **same process**.
- The biggest challenge when moving to **microservices-based** application is **changing the communication mechanism**.
- Microservices are **distributed** and microservices communicate with each other by **inter-service communication** on network level.
- Services must interact using an **inter-service communication** protocols like **HTTP**, **gRPC** or message brokers **AMQP** protocol.

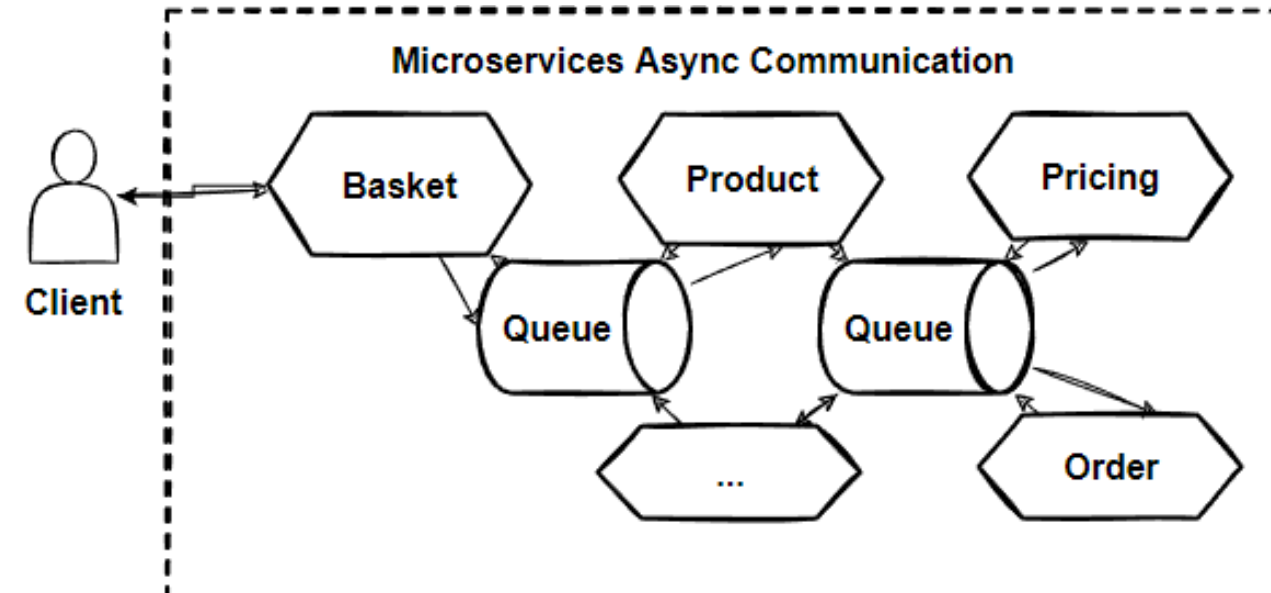
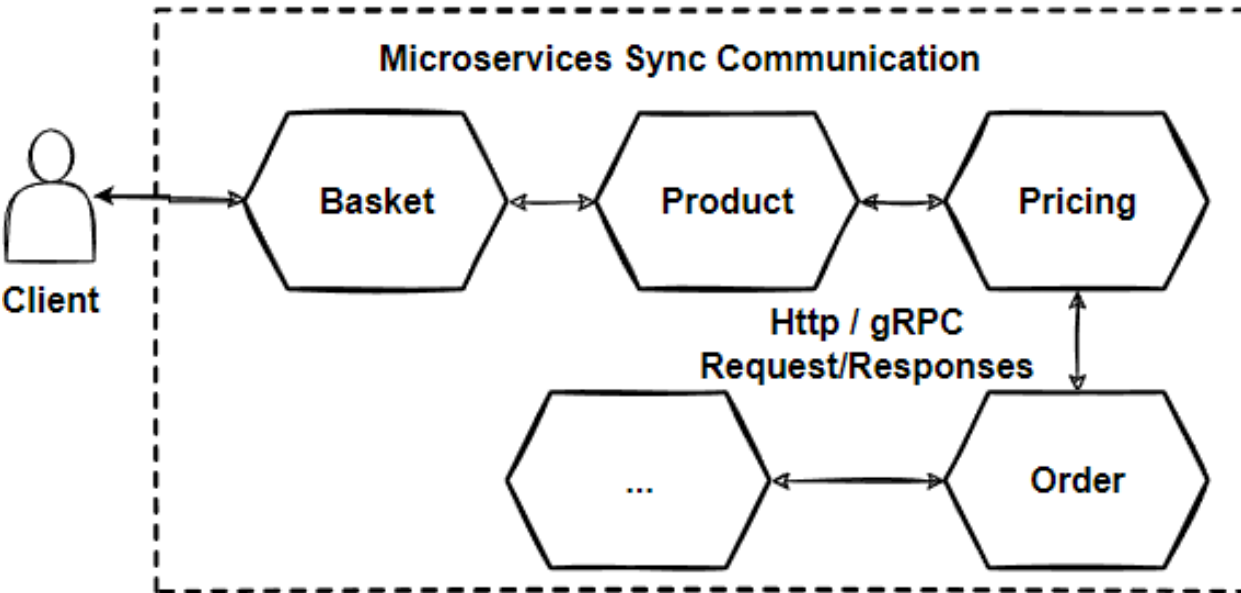
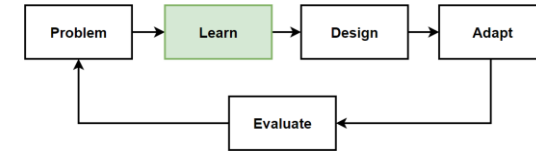


Microservices Communications

- **Isolate** the **business** into microservices as much as possible.
- Use **asynchronous communication** between the internal microservices as much as possible.
- Create **well-defined APIs** for **inter-service** communications.
- Monolithic inter-process method calls becomes **well-defined APIs** into Microservices.
- Group some operations and **expose aggregated APIs** that cover several calls from multiple sources.
- **Smart endpoints and dumb pipes:** microservices loosely coupling and expose endpoints with RESTFul APIs in order to provide end-to-end use cases.
- Microservices communication types:
 - **Synchronous communications:** Request/Response
 - **Asynchronous communications:** Message broker event buses

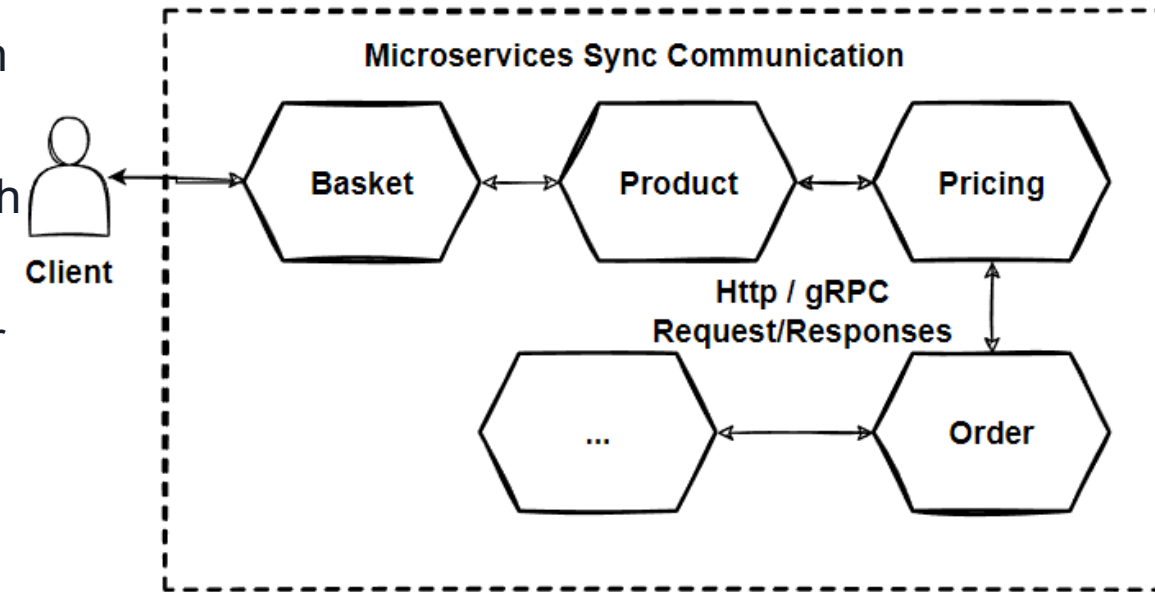
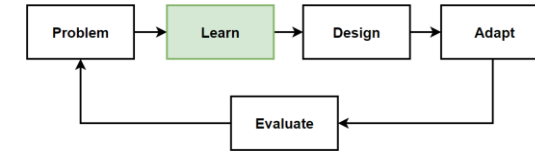


Microservices Communication Types - Sync or Async



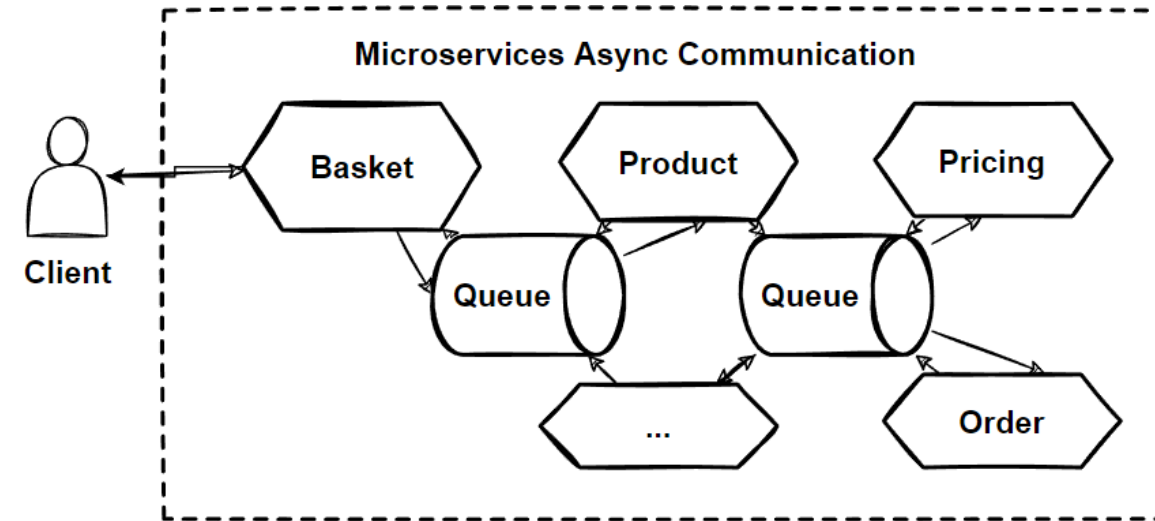
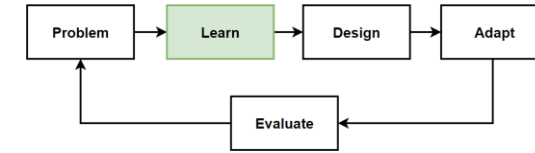
Microservices Synchronous Communication

- **Synchronous communication** is using **HTTP** or **gRPC** protocol for returning synchronous response.
- The client **sends a request** and **waits for a response** from the service.
- The client code **block their thread**, until the response reach from the server.
- The synchronous communication protocols can be **HTTP** or **HTTPS**.
- The client sends a request with using **http protocols** and waits for a response from the service.
- The client **call the server** and **block** client their operations.
- The client code will **continue** its task when it **receives** the HTTP server **response**.

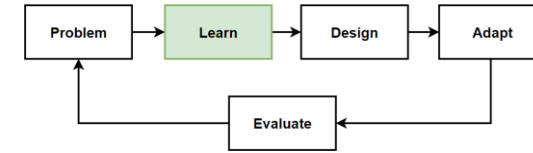


Microservices Asynchronous Communication

- The client sends a request but it **doesn't wait for a response** from the service.
- The client **should not have blocked** a thread while waiting for a response.
- **AMQP** (Advanced Message Queuing Protocol)
- Using AMQP protocols, the client **sends** the **message** with using message broker systems like **Kafka** and **RabbitMQ** queue.
- The message **producer does not wait** for a **response**.
- Message consume from the **subscriber** systems in async way, and no one waiting for response **suddenly**.
- **Asynchronous** communication also divided by 2:
 - one-to-one(queue)
 - one-to-many (topic)



Microservices Asynchronous Communication-2



- **one-to-one(queue) implementation**

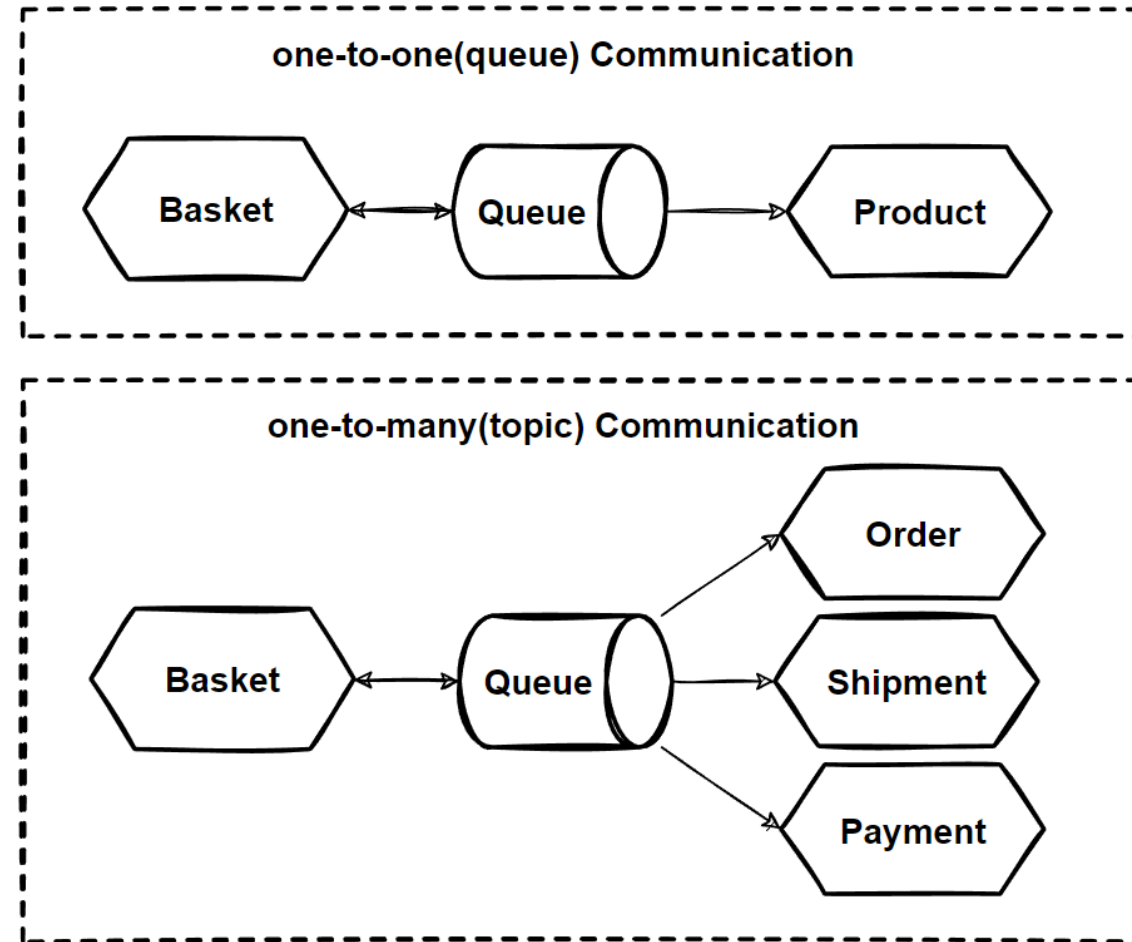
one-to-one(queue) implementation there is a single producer and single receiver.

- Command Patterns offers to receive one queue object and after that execute the command with incoming message. This process restarts with receiving new command queue item.

- **one-to-many (topic) implementation**

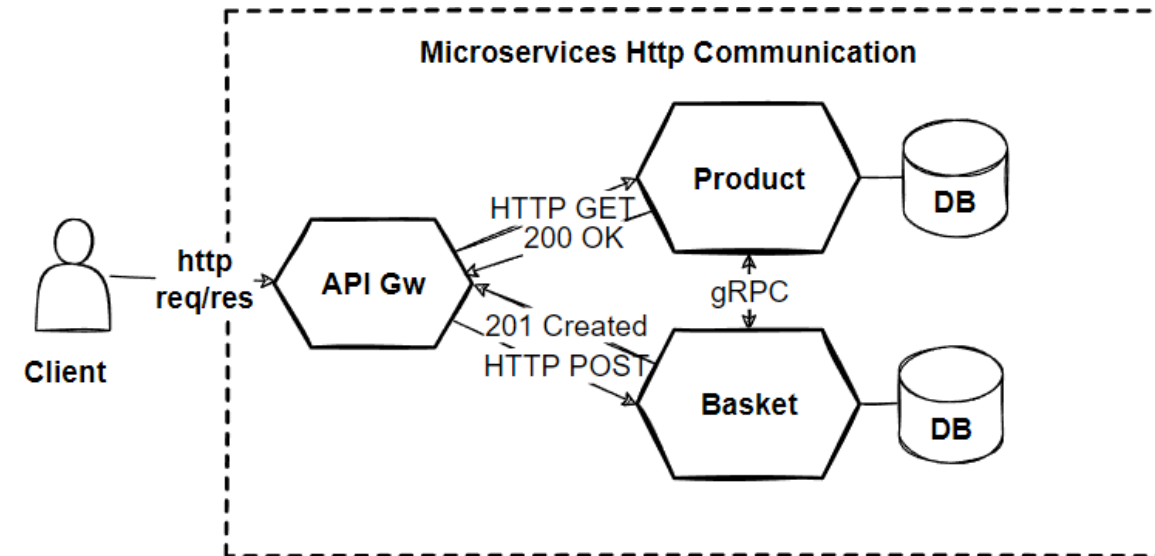
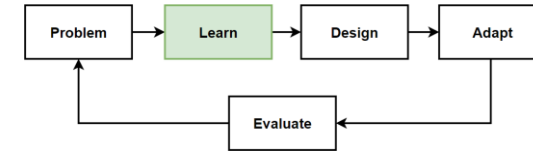
In one-to-many (topic) implementation has Multiple receivers. Each request can be processed by zero to multiple receivers.

- Event-bus or message broker system is **publishing events** between multiple microservices and communication provide with subscribing these events in an async way.
- **Publish/subscribe** mechanism used in Event-driven microservices architecture.



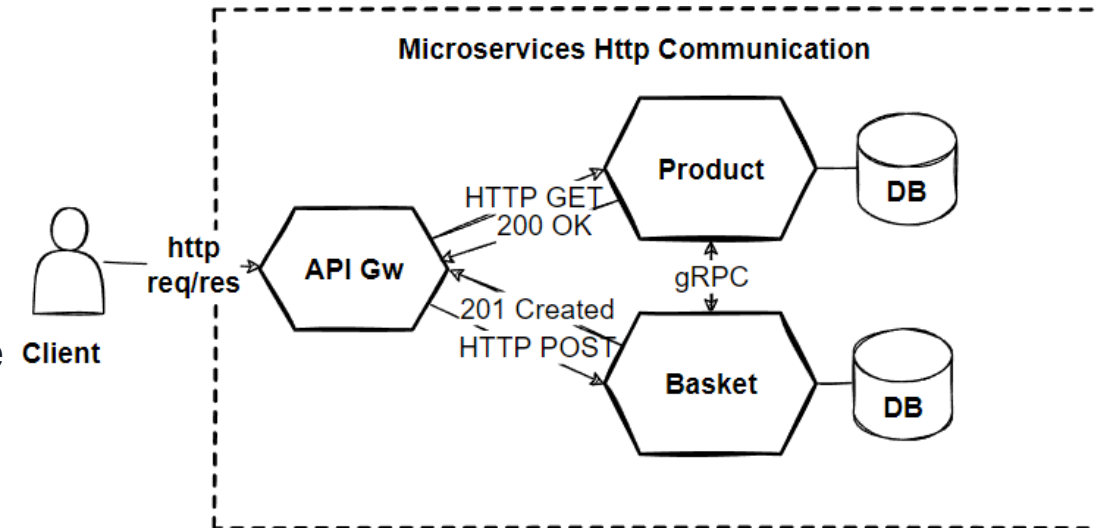
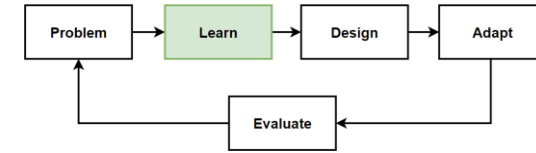
Microservices Communication Styles

- **Request/response communication** with HTTP and REST Protocol (extends gRPC and GraphQL)
- **Push and real-time communication** based on HTTP, WebSocket Protocol
- **Pull communication** based on HTTP and AMQP (short polling - long polling)
- **Event-Driven communication** with Publish/Subscribe Model

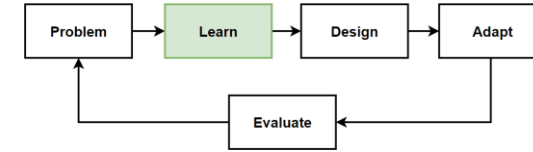


Request/response communication

- When using **synchronous** request/response-based communication mechanism, use **HTTP** and **REST** protocols that are the most common protocols.
- **Expose APIs** from our microservices using the HTTP and REST protocols.
- **REST HTTP calls** using HTTP verbs like GET, POST, and PUT.
- If our communication held between internal microservices, use **gRPC protocol** communication mechanisms to provide high performance and low latency.
- **Use GraphQL** instead of the REST APIs when performing Request/response communication.
- With GraphQL, we can define the **structure of the data** required and get 1 response in 1 request with more flexibility and efficiency way to get whole data.



Push/pull communication

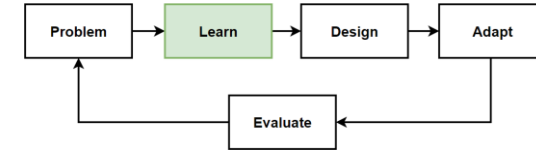


- **Push and real-time communication based on HTTP, WebSocket Protocol**

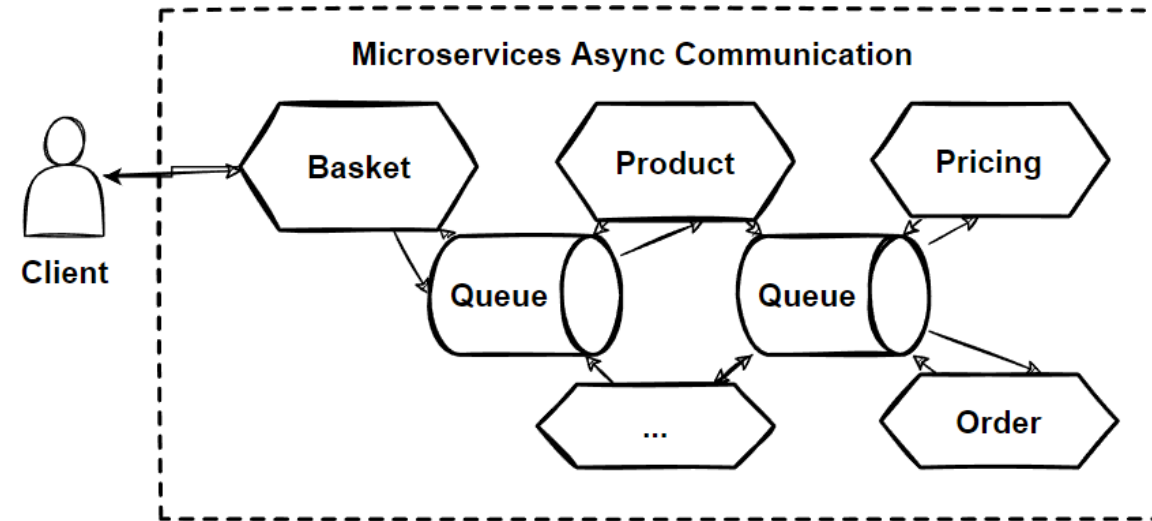
Use case about real-time and one-to-many communication like chat application, use Push Model with HTTP and WebSocket Protocols.

- Build real-time two-way communication applications, such as chat apps and streaming dashboards like the score of a sports game, with WebSocket APIs.
- The client and the server can both send messages to each other at any time. Backend servers can easily push data to connected users and devices.
- **Pull communication based on HTTP and AMQP (short polling - long polling)**
Also called "Polling" and it's basically the same as refreshing your mail inbox every 5 minutes to check for new mail. It is a call and ask model.
- This model is become a waste of bandwidth if there are no new messages and responses comes from the server.
- Opening and closing connections is expensive. And we can say that this model doesn't scale well.
- Typically have limits like on Twitter on how often they allow you to call their API.

Event-Driven communication with Publish/Subscribe

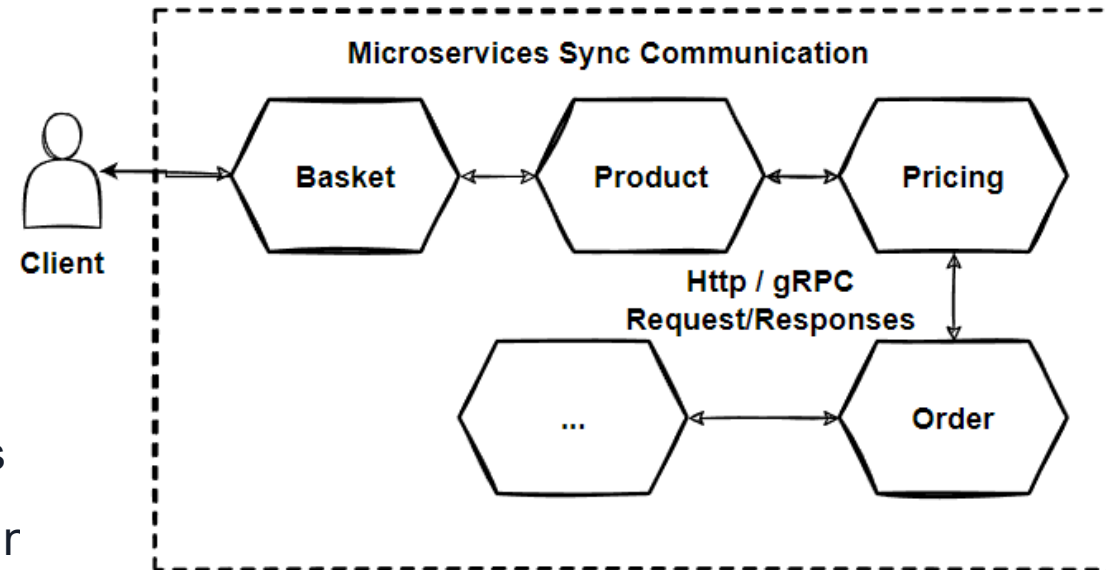
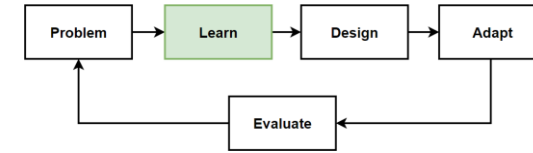


- Microservices don't call each other, instead, they created **events** and **consume events** from message broker systems in an async way.
- **AMQP** (Advanced Message Queuing Protocol)
- **Publish/Subscribe pattern** with events
- Using AMQP protocols, the client sends the message with using message broker systems like **Kafka** and **RabbitMQ** queue.
- The producer service of the events **doesn't know** about its consumer services. the consumers also don't necessarily know about the producer.
- **Decouple services** and build loosely coupled microservices.
- No clear central place or orchestrator, This increase **complexity** of architecture.

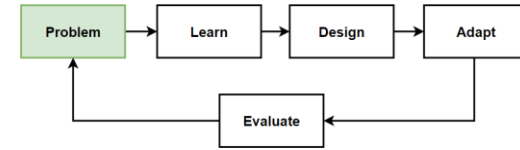


Microservices Synchronous Communications and Best Practices

- The client **sends a request** with using http protocols and **waits for a response** from the service.
 - The synchronous communication protocols can be **HTTP** or **HTTPS**.
 - **Request/response communication** with HTTP and REST Protocol (extends gRPC and GraphQL)
 - **REST HTTP APIs** when exposing from microservices
 - **gRPC APIs** when communicate internal microservices
 - **GraphQL APIs** when structured flexible data in microservices
 - **WebSocket APIs** when real-time bi-directional communication
-
- How can we design and exposing APIs with HTTP protocols for our microservices ?



Problem: Direct Client-to-Service Communication



Problems

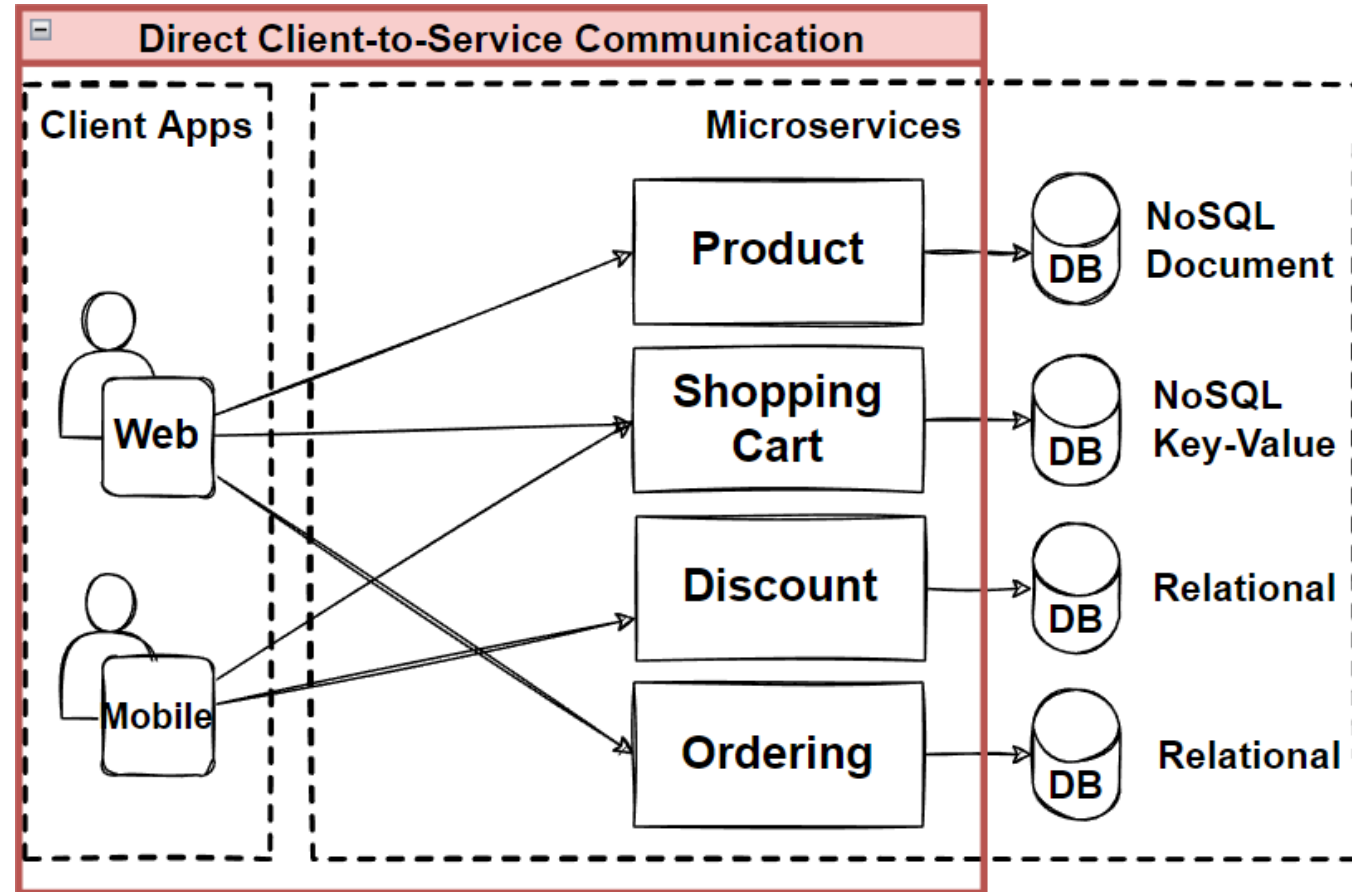
- Direct Client-to-Service Communication
- Cause to chatty calls from client to service
- Hard to manage invocations from client app.

Solutions

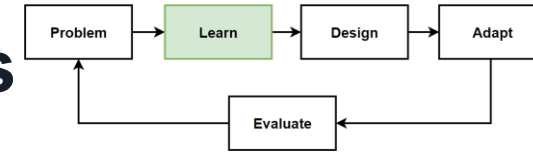
- Well-defined API Design
- Microservices Communication Patterns

Steps

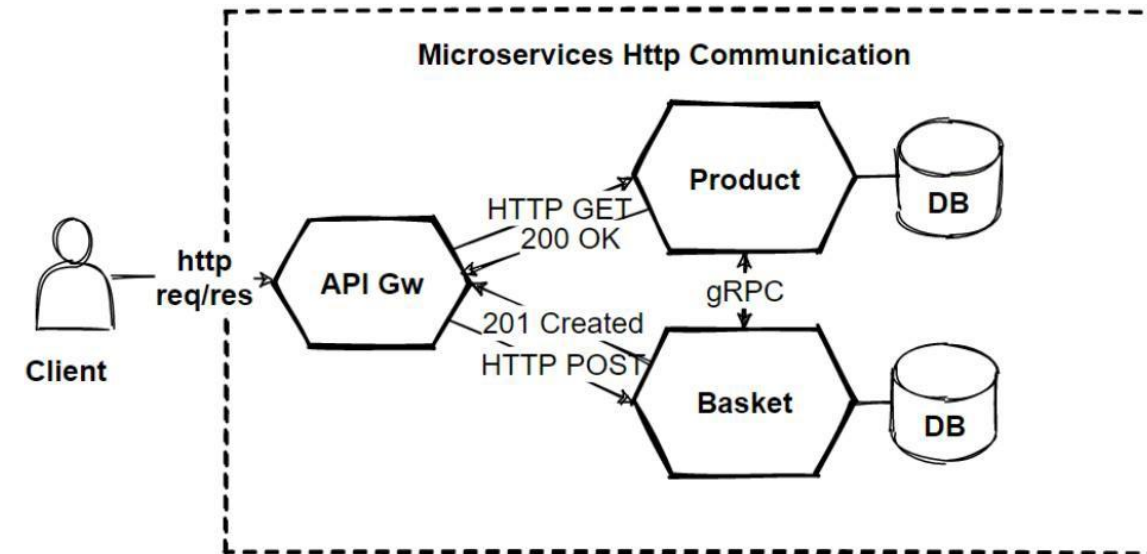
- Microservice Communications and Types
- Well-defined RESTful API Design for services
- Microservices Communication Patterns (API Gateway, BFF, Publish/Subscribe..)



Designing HTTP based RESTful APIs for Microservices

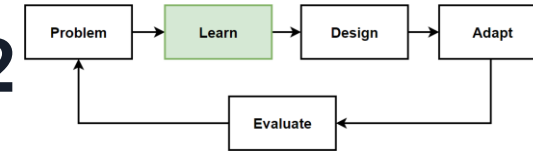


- When making **request/response communication**, we should use **REST** when designing our APIs. Its also called **Restful APIs**.
- **REST** approach is following the **HTTP protocol**, and implementing HTTP verbs like GET, POST, and PUT.
- **REST** is the most commonly used architectural communication approach when creating **APIs** for our **microservices**.
 - Java and Sprint Boot framework
 - C# with ASP.NET Core WebAPI
 - Python with Flask Web API framework

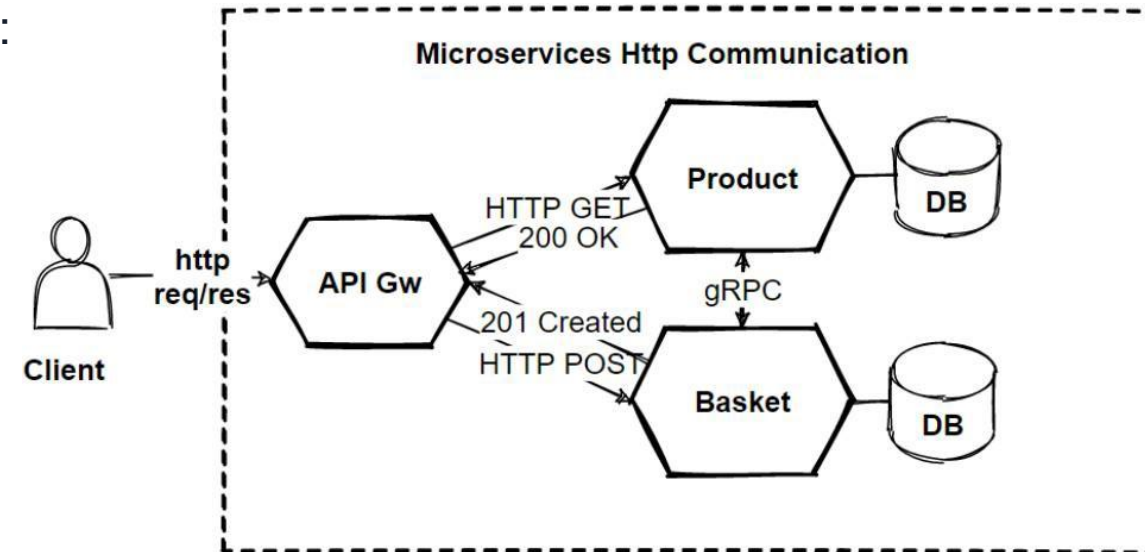


- How to **design** our **APIs** for **microservices** ?

Designing HTTP based RESTful APIs for Microservice2

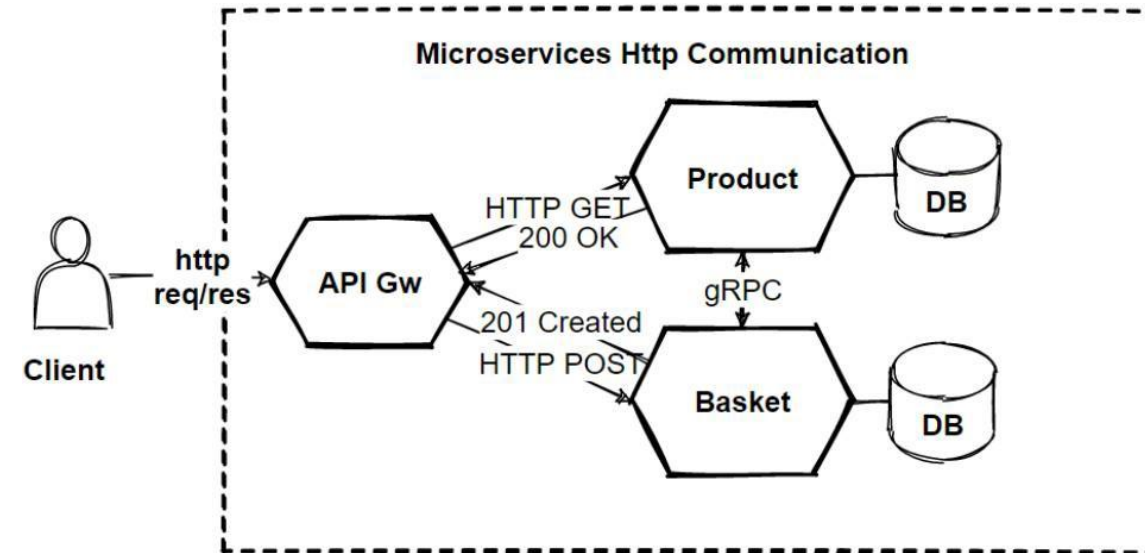
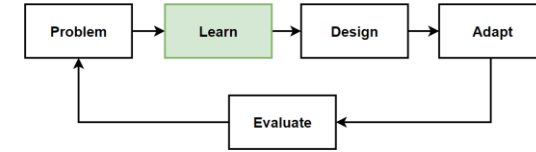


- **Well-defined API** design is very important in a microservices architecture, communication happens API calls.
- Designed APIs should be **efficient** and **not** to be **chatty communications**. APIs must have well-defined documented and versioning.
- There are 2 type APIs sync communication in microservices:
 - Public APIs: API calls from the clientapps.
 - Backend APIs: inter-service communication between backend services.
- **Public APIs**
Use RESTful APIs over HTTP protocol. RESTful APIs use JSON payloads for request-response, that easy to check payloads and easy agreement with clients.
- **Backend APIs**
Inter-service communication can result in a lot of network traffic. serialization speed and payload size become more important. Using gRPC is mandatory for increase network performance.



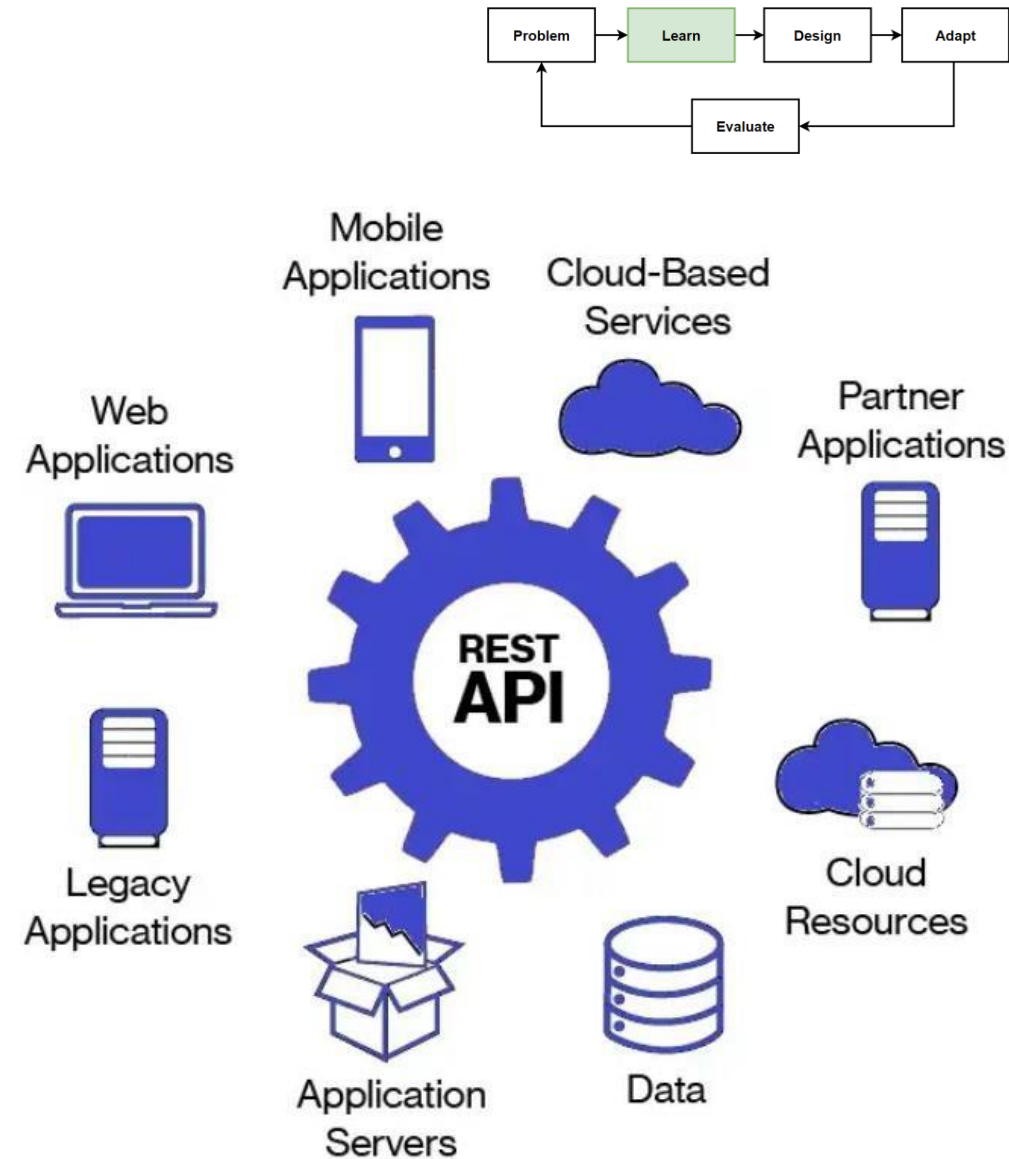
Comparison with REST and gRPC

- **REST** is using HTTP protocol, and request-response structured JSON objects.
- API interfaces design based on HTTP verbs like GET-PUT-POST and DELETE.
- **gRPC** is basically Remote Procedure Call, that basically invoke external system method over the binary network protocols.
- Payloads are **not readable** but its **faster** that **REST APIs**.
- 2 Main Approaches for Public and Backend APIs:
 - RESTful API Design over HTTP using JSON
 - gRPC binary protocol API Design



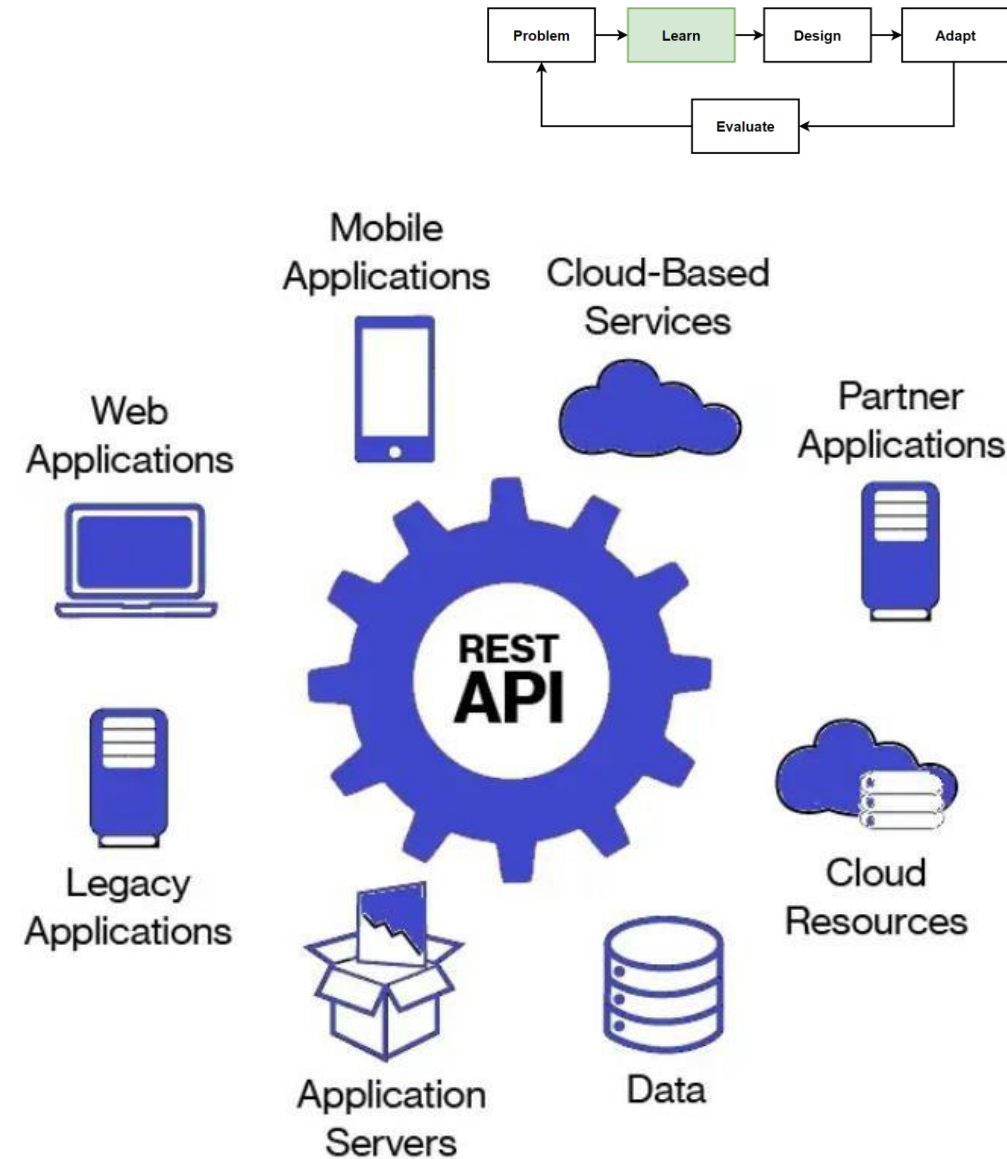
RESTful API design for Microservices

- **RESTful services** are widely used in modern Web architectures.
- It is lightweight, extensible and simple easy develop services.
- **What is REST ?**
REST (Representational State Transfer) is a service structure that enables easy and fast communication between client and server. Roy Fielding introduced and developed REST in his doctoral thesis in 2000.
- REST allows applications to communicate with each other by **carrying JSON data** between the client and server.
- **Characteristics of the REST Architecture:**
 - Stateless
 - Uniform Interface
 - Cacheable
 - Client-Server
 - Layered System
 - Code on Demand

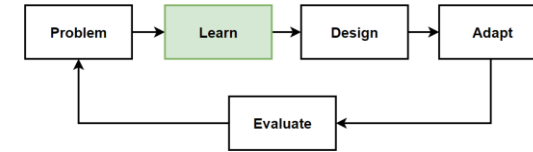


What is RESTful APIs?

- Web services that use REST architecture are called **RESTful services**.
- **RESTful systems** communicates over **HTTP protocol** with HTTP methods used by Web Browsers to transfer pages.
- **Richardson Maturity Model**
Leonard Richardson proposed the Richardson Maturity Model for web APIs:
 - **Level 0:** Define one URI, and all operations are POST requests to this URI.
 - **Level 1:** Create separate URIs for individual resources.
 - **Level 2:** Use HTTP methods to define operations on resources.
 - **Level 3:** Use hypermedia

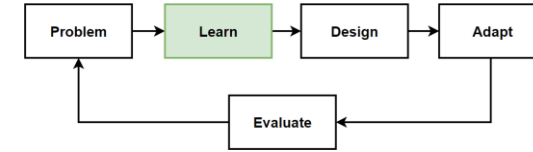


HTTP Methods

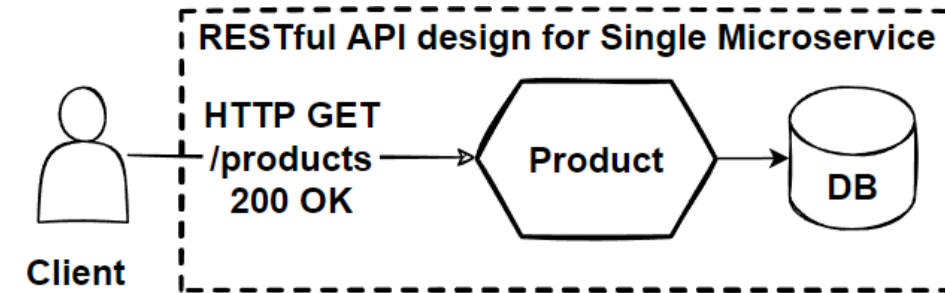


HTTP Methods		
GET	Retrieve information	
HEAD	Retrieve Resource Header	
POST	Submit Data to the server	
PUT	Save an object at the location	
DELETE	Delete the object at the location	
PATCH	Update single piece of data	

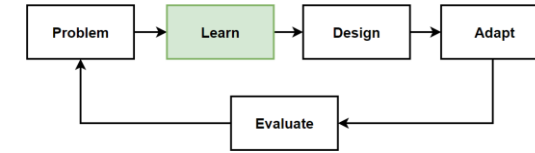
RESTful API design for Single Microservice



- **Focus on the business entities** that we expose APIs for our application. Organize our resources according to business entities and expose via APIs.
- Design RESTful APIs for single **Product Microservices**.
- **Rule1- REST APIs are designed around "resources"**
The best practice is the resource URIs should be based on nouns = the resource and not verbs. REST API Url: <https://e-shop.com/products>
 - <https://e-shop.com/products> -- Correct
 - <https://e-shop.com/create-product> -- Wrong
- **Rule2- Every Resource has an identifier**
Unique identifiers for that resource. REST API URL becomes for a particular product: <https://e-shop.com/products/4>
- This will return single product which id = 4.
- API frameworks route requests based on parameterized URI paths, so we can define a route for the path like **/products/{id}**.



RESTful API design for Single Microservice-2



- **Rule3- REST APIs are using JSON**

REST API url has GET request, then it will return this response body as a JSON response:

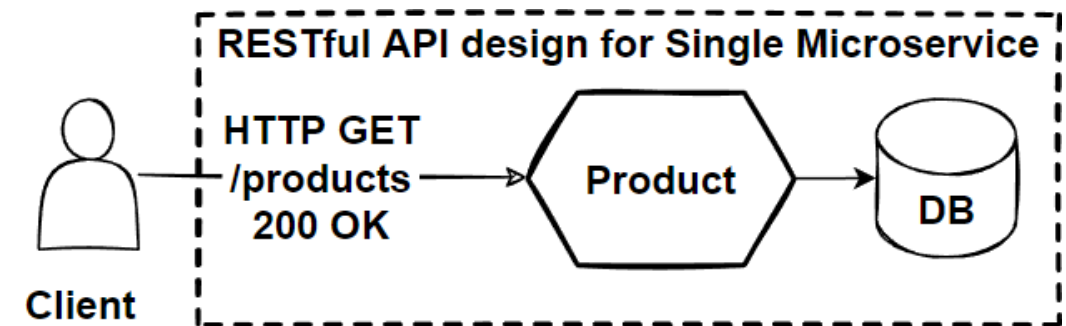
- `{"productId":1,"name":"iPhone", "price":44.90,"category":"Electronics"}`

- **Rule4- REST APIs perform operations on Resources with HTTP verbs:**

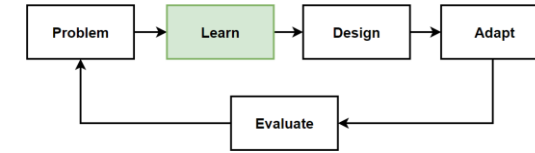
REST APIs built on HTTP protocol and perform operations on Resources with HTTP verbs, these verbs are GET, POST, PUT, PATCH, and DELETE.

- When creating a product, we send HTTP POST request with contains product detail informations and return back to HTTP response that including 200 OK success response.

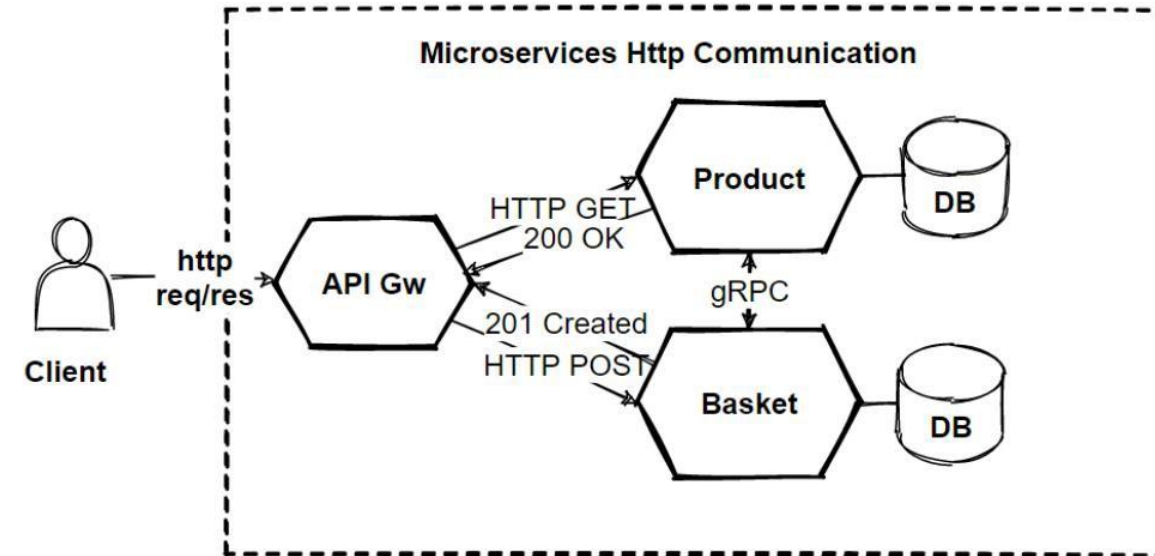
- Request:
- HTTP: POST
- Url: `https://e-shop.com/products`
- Body: `{"productId":1,"name":"iPhone"}`
- Response: 201 Created



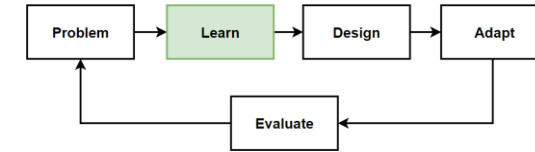
RESTful API design for E-Commerce Microservices



- RESTful API design for E-Commerce Microservices: Customer, Order and Product resources.
- **How should we design Rest API when you required to get customer orders ?**
 - URL: <https://e-shop.com/customers/6/orders>
- I want to retrieve products into order number 22 which ordered by customer 6:
 - URL: <https://e-shop.com/customers/6/orders/22/products> --**WRONG**
- This level of complexity can be difficult to maintain.
- Instead, we should keep URIs relatively simple.
- Resource URIs should not be more complex than "**collection/item/collection**" path.
- First we should get Customer orders with this API:
 - URL: <https://e-shop.com/customers/6/orders>
- After that we should locate the Order no 22 and get all products:
 - URL: <https://e-shop.com/orders/22/products>

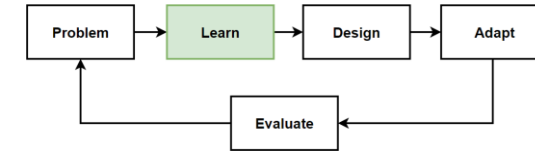


RESTful API design with HTTP Methods

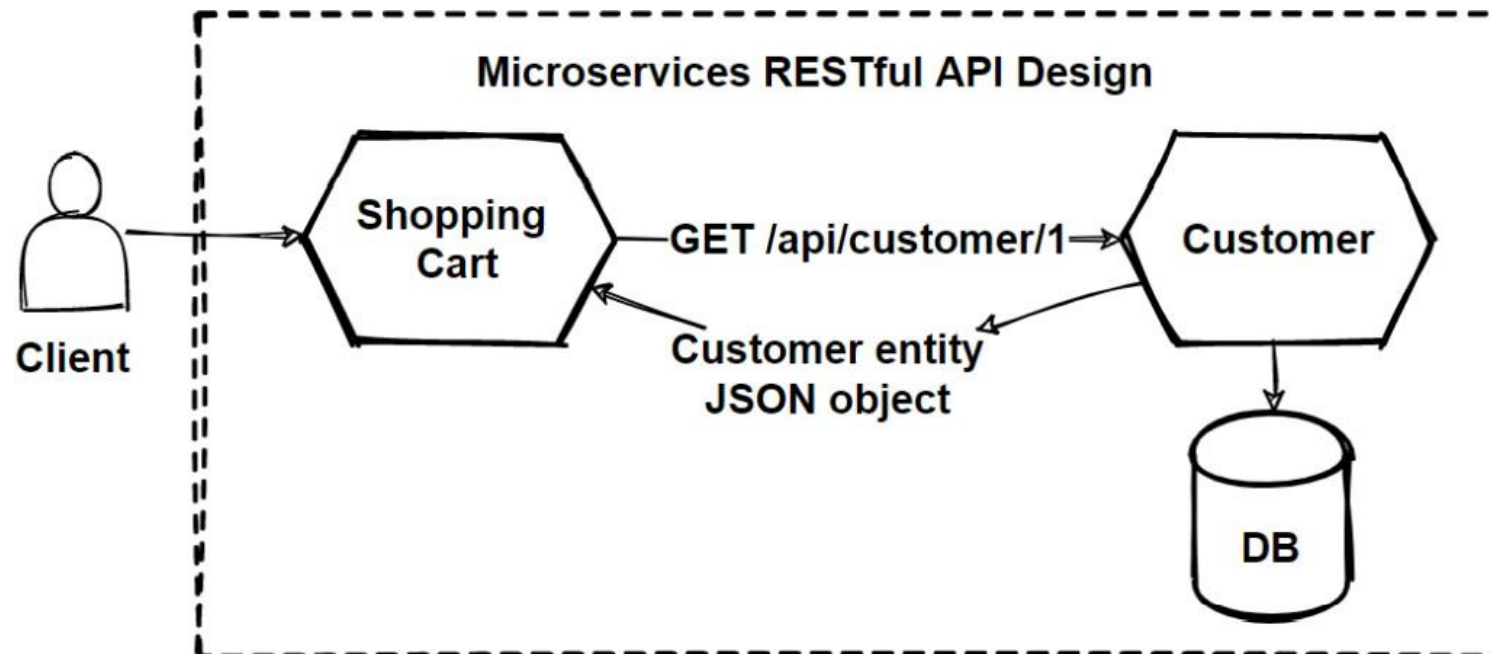


Resource	POST	GET	PUT	DELETE
/customers	Create new customer	Retrieve all customers	Update all customers	Remove all customers
/customers/1	N/A	Retrieve data for customer 1	Update data for customer 1 if exist	Remove customer 1
/customers/1/orders	Create new order for customer 1	Retrieve all orders for customer 1	Update all orders for customer 1	Remove all orders customer 1

RESTful API design for Relational Data

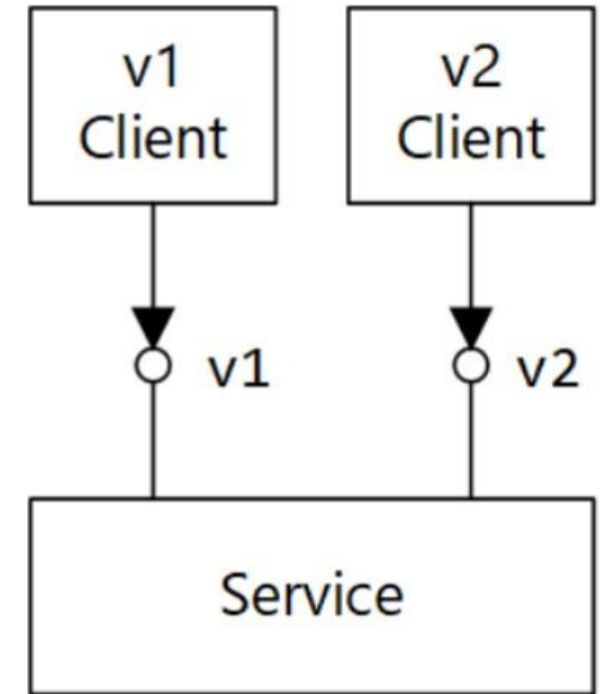
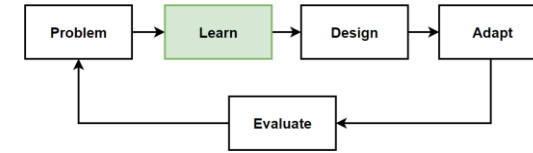


- Microservices don't share the same code base and **don't share data stores**. They communicate **through APIs** for data operations.
- Shopping Cart service **requests information** about a customer from the Customer service.
- Customer service **retrieve** data with using Repository classes and return **Customer entity model** as a **JSON object** in an HTTP response.



API versioning in Microservices RESTful APIs

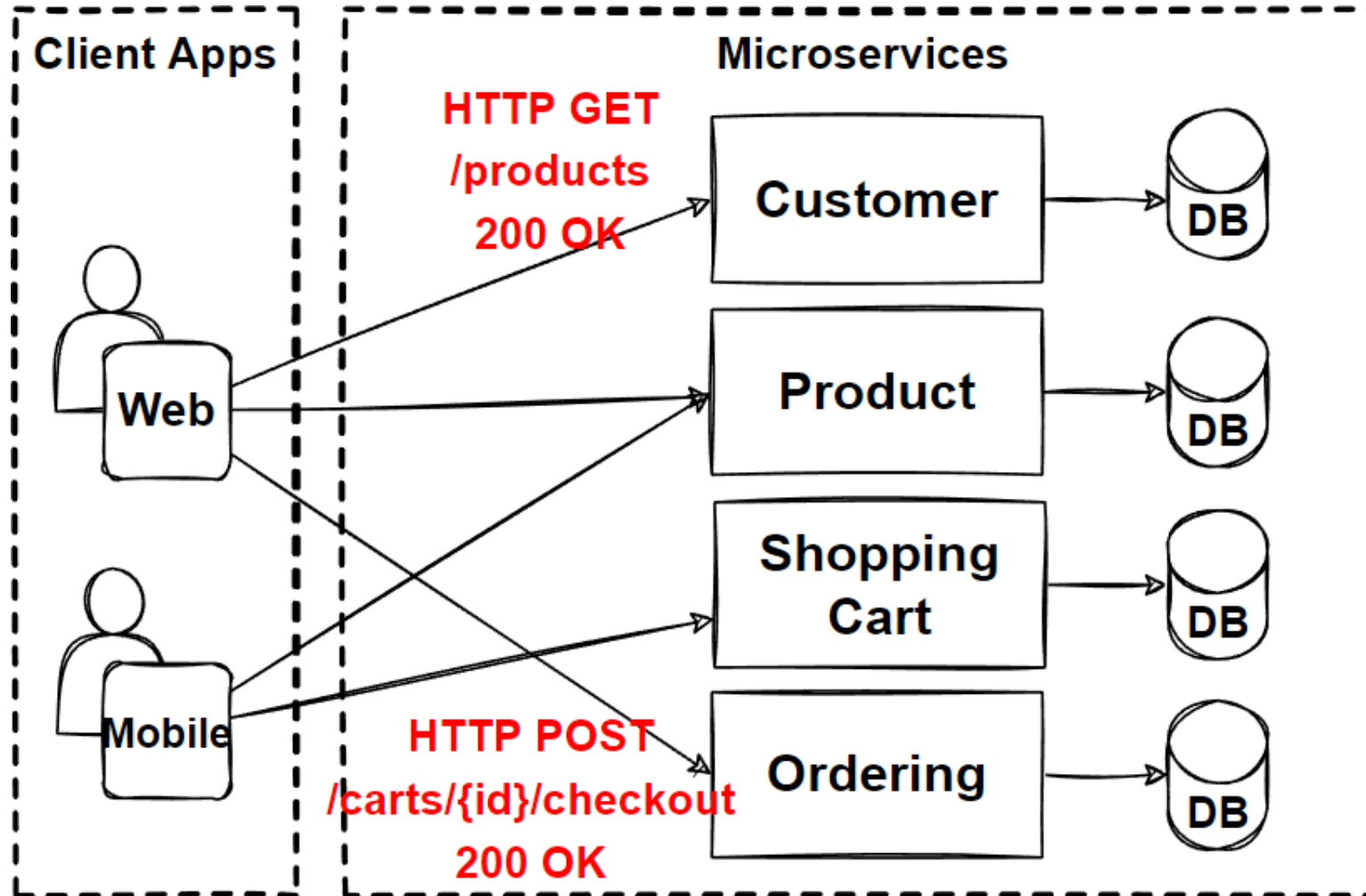
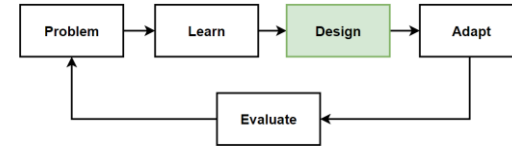
- If an **API changes**, there may broke some communications which depending on that API from other microservices.
- Services may required to add **new features** or **bug fixes** that require changing an existing API.
- API changes should be **backward compatible** and **not break** any communications.
- Best practice for the **roll-out** or **canary deployments** on **Kubernetes**.
- If your microservices has one more pods, new version of API deployments handle one by one and if there is a problem its **easy** to **rollback** to previous version.
- Microservices should **supports two versions** with following backward compatible.



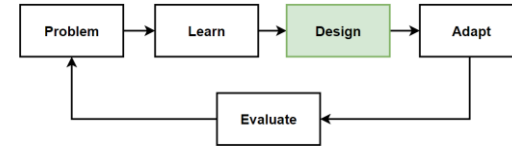
Before Design – What we have in our design toolbox ?

Architectures	Patterns&Principles	Non-FR	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications • HTTP Based RESTful API design	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Design: Microservices Architecture with RESTful APIs

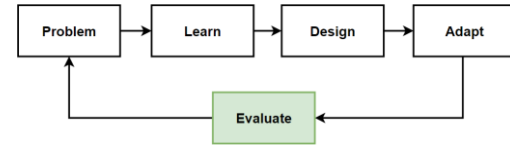


Design: Microservices Architecture with RESTful APIs



Microservices	Resource	HTTP Methods	REST APIs	
Customer	/customers	HTTP GET HTTP POST HTTP DELETE	GET /customers GET /customers/{id} POST /customers DELETE /customers/{id}	
Product	/products	HTTP GET HTTP POST HTTP DELETE	GET /products GET /products/{id} POST /products DELETE /products/{id}	
Shopping Cart	/carts	HTTP GET HTTP POST HTTP DELETE	GET /carts GET /carts/{id} POST /carts/{id}/checkout DELETE /carts/{id}	
Ordering	/orders	HTTP GET HTTP POST HTTP	GET /orders GET /orders/{id}	

Evaluate: Microservice Architecture with RESTful APIs

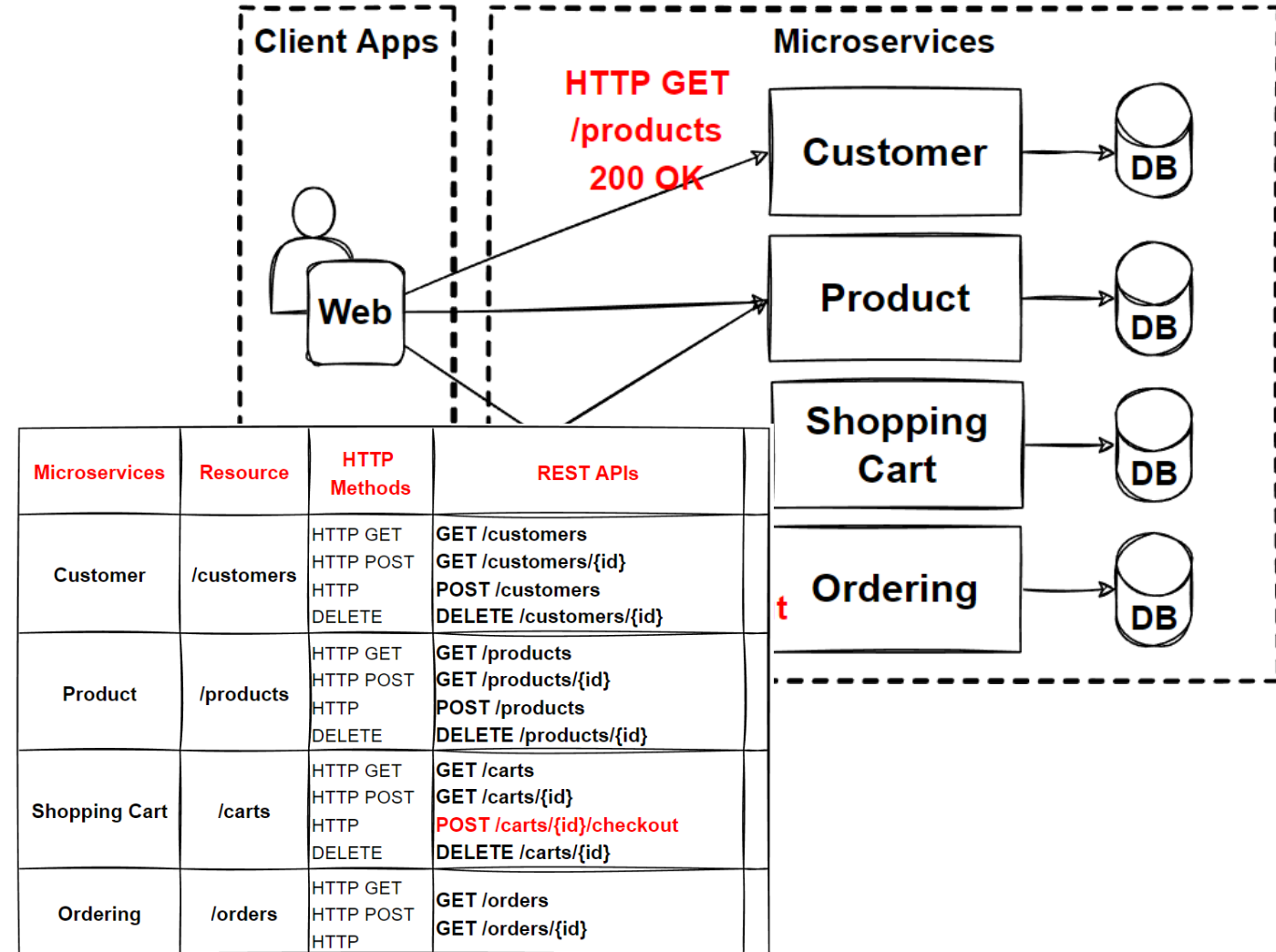


Benefits

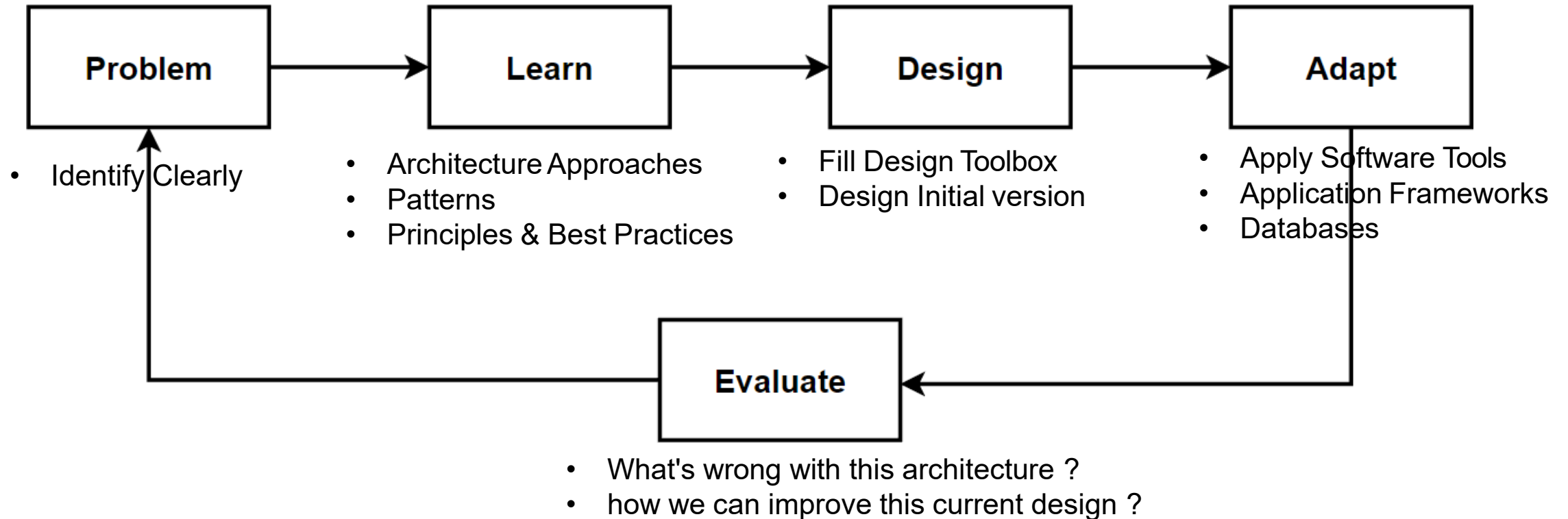
- Request can simply sent using browsers.
- HTTP protocol
- HTTP GET has organic caching options
- JSON representative request-responses

Drawbacks

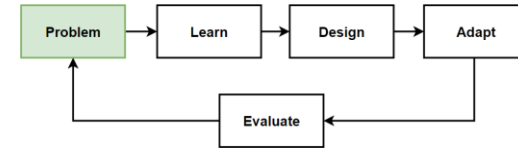
- Send multiple request to get relational data
- Chatty communication when enriching data



Way of Learning – The Course Flow



Problem: Multiple Request for Retrieve Relational Data

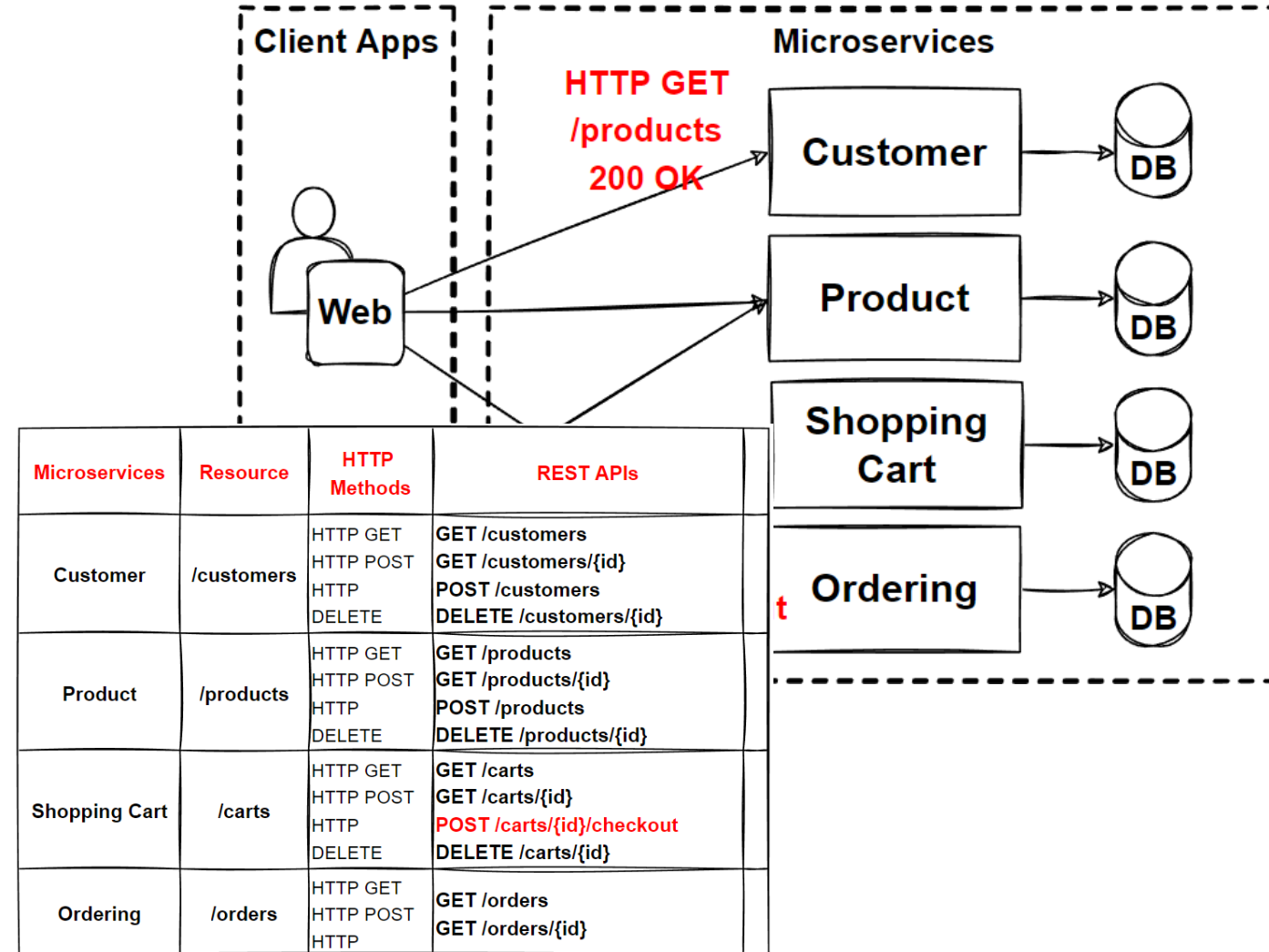


Problems

- Business teams want to see relational data on screen. N+1 problem.
- Customer – Orders – Products
- REST: /customer/3/orders/4/products ?
- How we can get Products from Order X for Customer Y ?
- Chatty Calls for enriching data

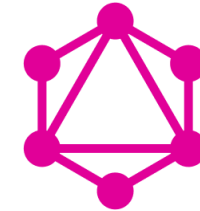
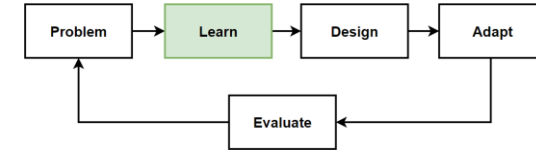
Solutions

- GraphQL API Design
- Structural relational data with querying GraphQLAPI



GraphQL: A query language for APIs

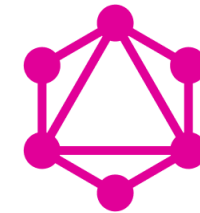
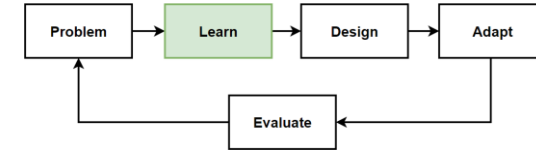
- **GraphQL** is a **query** and **manipulation** language for APIs and a runtime for fulfilling queries with your existing data.
- It was developed internally by **Facebook** in 2012 before being publicly released in 2015.
- It allows **clients** to **define** the **structure** of the **data required**, and the **same structure** of the data is returned from the server.
- It provides a **complete** and **understandable description** of the data in our API, clients request what they need, **easier** to **evolve** APIs over time, has powerful developer tools.
- It provides a **flexible syntax** to describe data requirements and interactions.
- It provides to **access many sources** in a single request, **reducing** the **number of multiple network** calls.
- Developers to ask for **exactly what is needed** and get back **predictable results**.



GraphQL

Characteristics of GraphQL

- **Ask for what you need, get exactly that**
Send a GraphQL query to your API and get exactly what you need. GraphQL queries always return predictable results.
- **Get many resources in a single request**
GraphQL queries can access the properties of one resource and also relations of particular resource references between them.
- GraphQL APIs get all the data your applications needs in a single request.
- **Evolve APIs without versions**
As you know that REST APIs requires versions when change your resource properties. We can add new fields and types to GraphQL API without impacting existing queries.
- By using a single evolving version, GraphQL APIs give apps continuous access to new features.

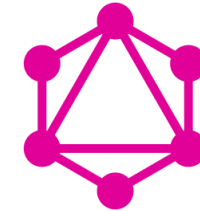
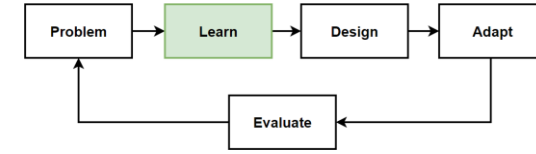


GraphQL

Core Concepts of GraphQL

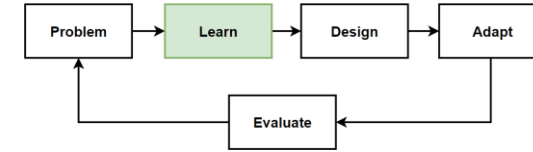
Schemas, Queries, Mutations and Resolvers

- Developers can use **GraphQL** in order to create a **schema**, which describes all the possible data that clients can query on it.
- **GraphQL** schema is made up of **object types**, that define which kind of object you can request and which fields it has.
- Client can **send queries**, GraphQL validates the queries against the schema then executes the **validated queries**.
- **Resolver** is a function that attaches to fields in a schema. During execution, the resolver is called to produce the value.
- **Mutation** is a GraphQL Operation that allows you to **insert new data** or **modify the existing data** on the server-side.



GraphQL

Example of GraphQL Query



- Simply **send a single query** to the GraphQL server that includes the **concrete data** requirements. The server then responds with a JSON object.
- See example **Order** and **Order Products** list as below:

POST Request:

```
{
  orders {
    id
    productList {
      product {
        name
        price
      }
      quantity
    }
    totalAmount
  }
}
```

Response:

```
{
  "data": {
    "orders": [
      {
        "id": 1,
        "productList": [
          {
            "product": {
              "name": "orange",
              "price": 1.5
            },
            "quantity": 100
          }
        ],
        "totalAmount": 150
      }
    ]
  }
}
```

Advantages of GraphQL

- **GraphQL is fast**

GraphQL is way faster than other communication APIs like REST APIs because it reduce to multiple calls to get data and provides to query by choosing only the specific fields.

- **Single Request**

GraphQL get all the data your applications needs in a single request. Clients get what they request with no over-fetching.

- **Strongly typed**

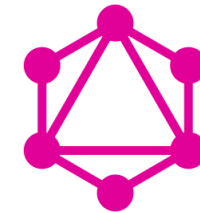
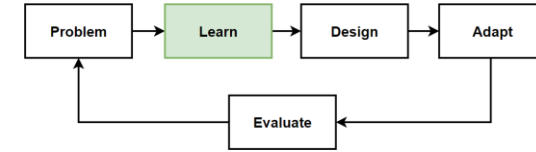
In GraphQL we describe data before query it. Strongly defined data types reduce miscommunication.

- **Hierarchical Structure**

GraphQL provides a hierarchical structure where relationships between objects are defined in a graphical structure.

- **Evolve API**

GraphQL allows an application API to evolve without breaking existing queries.



GraphQL

Disadvantages of GraphQL

- **GraphQL Query Complexity**

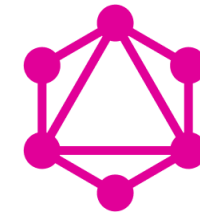
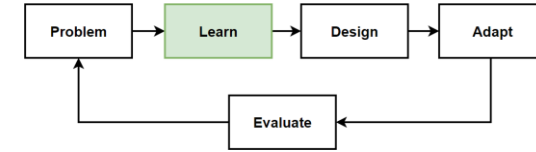
When access multiple fields in one query, it may requests too many nested fields data at a single time that could cause performance problems. Consider maximum query depths, query complexity.

- **GraphQL Caching**

More complicated to implement a simple cache with GraphQL. Complex because each query can be different, even though it operates on the same entity.

- **GraphQL Rate Limiting**

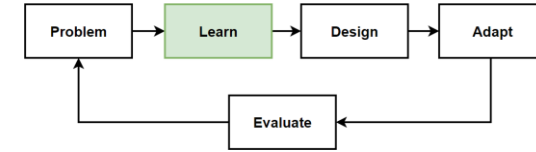
In REST API, we can simply specify that we allow only the specific amount of requests in specific time, but In GraphQL, it is difficult to specify this type of statement.



GraphQL

REST vs GraphQL APIs

- **REST** has become the de-facto standard for designing web APIs, However, REST APIs are too inflexible to rapidly changing requirements of the clients.
- **GraphQL** was developed for more flexibility and efficiency that solves many of the shortcomings and inefficiency calls.
- **REST** has the advantages of being a popular API, and also it is easy to understand, as well as scalable.
- **GraphQL** is more complex than REST, main drawbacks are error reporting, caching, and N+1.
- **GraphQL** solves the under-fetching problem that REST can't fetch hierarchical data at once.
- **GraphQL** and **REST** are two API styles that have a different approach when dealing with the transfer of data over HTTP protocols.



REST vs GraphQL APIs

REST APIs

- Easy to understand
- Defacto standard Popular APIs for designing web APIs
- Provide better scalability
- Built-in caching that minimizes multiple trips

Drawbacks:

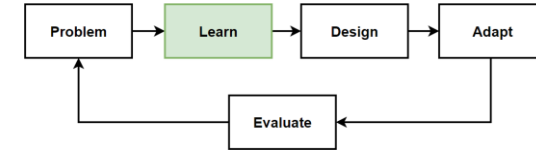
- Over-fetching and under-fetching requests are hard to implement and required multiple calls.

GraphQL APIs

- Solving the over and under-fetching problem
- More flexible with Schema and Type System
- Quick response time
- Protocol agnostic

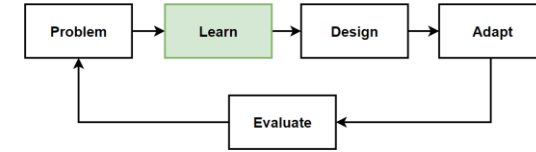
Drawbacks:

- Complexity and caching problems.



Fetching Data with REST and GraphQL

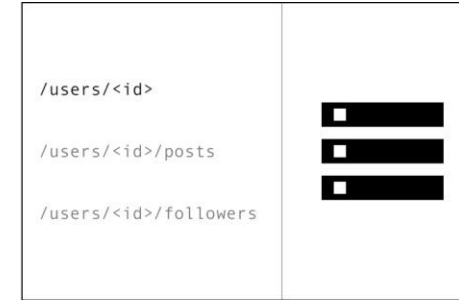
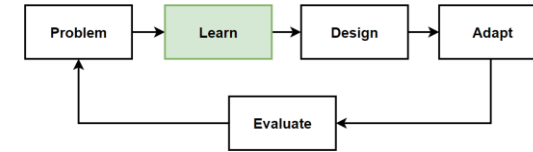
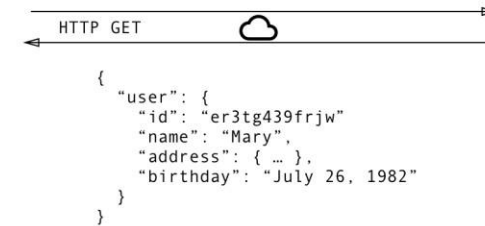
- **Blogging application**, an application needs to display screen,
- **"posts"** of the specific **"user"** and the last 3 **"follower"** names at the same screen.
- users -> posts, users -> followers
- How can we fetch this data with REST and GraphQL ?



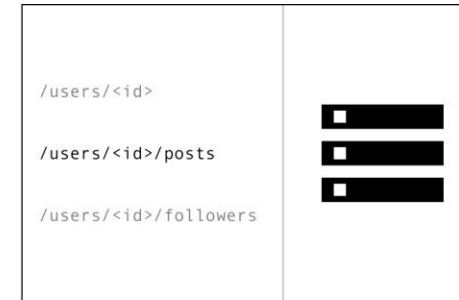
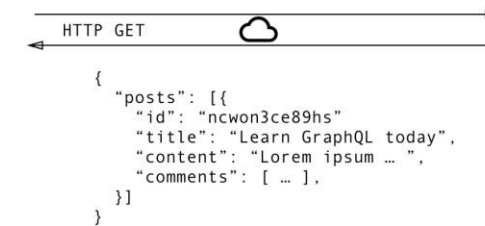
Fetching Data with REST APIs

- **REST API**, we have to call multiple endpoints to fetch data.
 1. `/users/<id>` endpoint to fetch the initial user data.
 2. `/users/<id>/posts` endpoint that returns all the posts for a user.
 3. `/users/<id>/followers` that returns a list of followers per user.
- With REST, have to **make 3 requests** to different endpoints to fetch the required data.
- It is **over-fetching** since the endpoints return additional information that's not required.

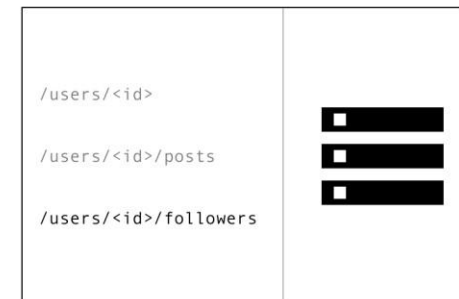
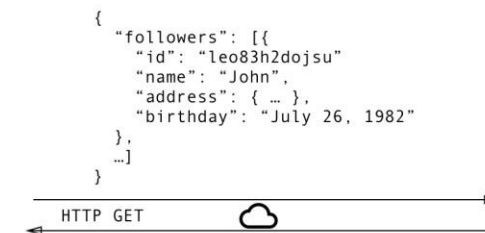
1



2

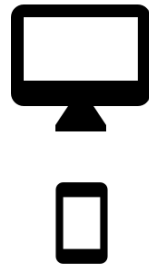


3



Fetching Data with GraphQL APIs

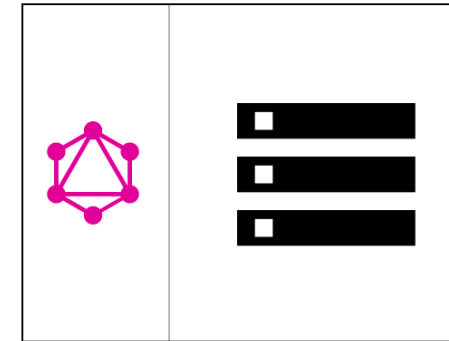
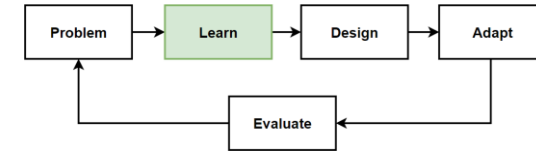
- **GraphQL API**, simply **send 1 single** query to the GraphQL server that includes the concrete data requirements.
- Then the server responds with a **JSON object** that includes required fields.
- Using GraphQL, the client can specify **exactly the data** it needs in a query.
- The structure of the server's response follows **precisely the nested structure** defined in the query.



```
query {  
  User(id: "er3tg439frjw") {  
    name  
    posts {  
      title  
    }  
    followers(last: 3) {  
      name  
    }  
  }  
}
```

HTTP POST

```
{  
  "data": {  
    "User": {  
      "name": "Mary",  
      "posts": [  
        { title: "Learn GraphQL today" }  
      ],  
      "followers": [  
        { name: "John" },  
        { name: "Alice" },  
        { name: "Sarah" },  
      ]  
    }  
  }  
}
```

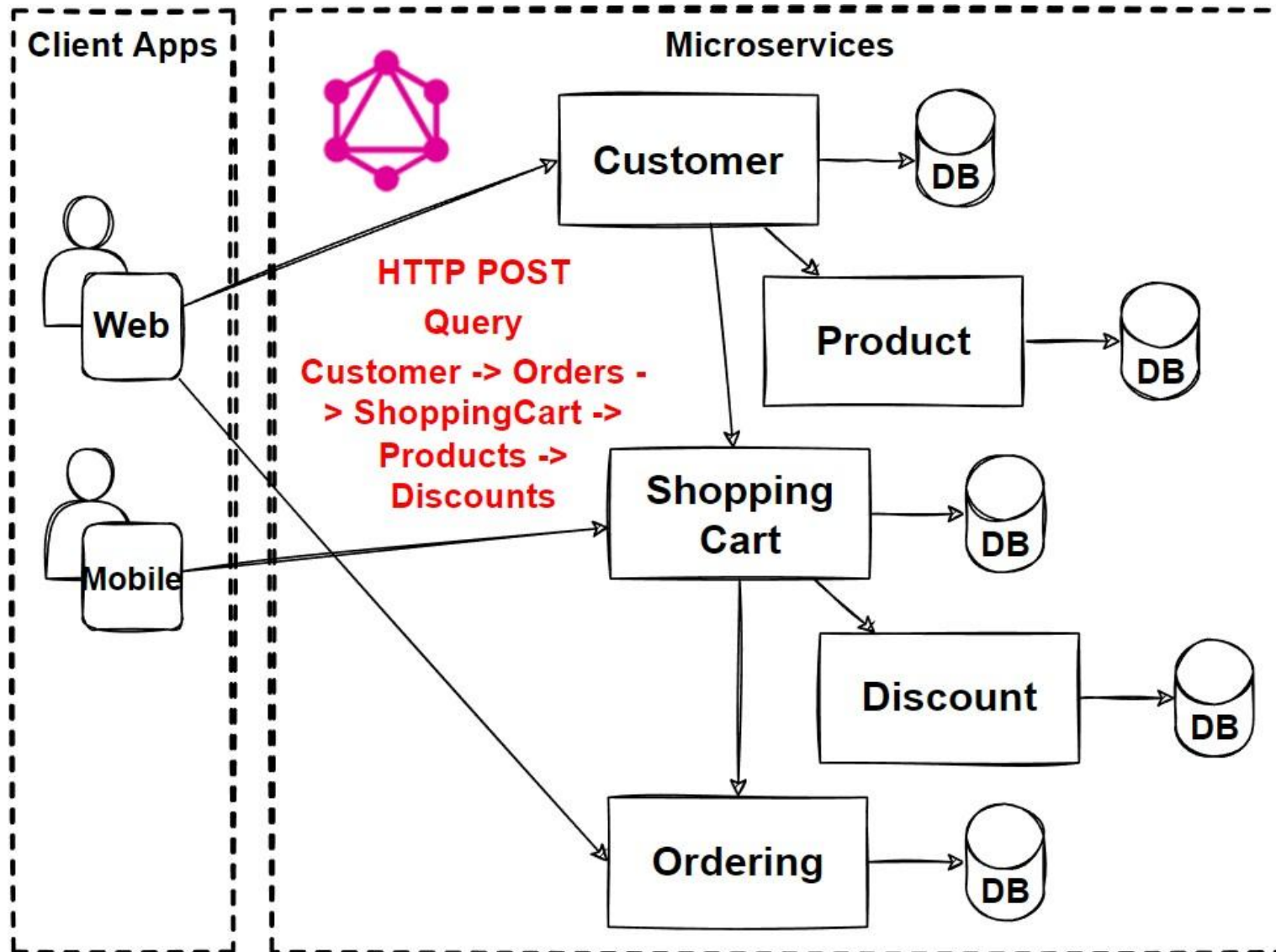
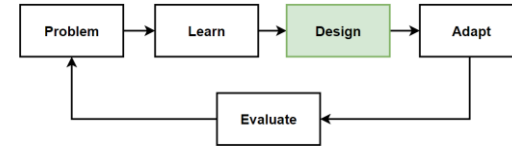


<https://www.howtographql.com/basics/1-graphql-is-the-better-rest/>

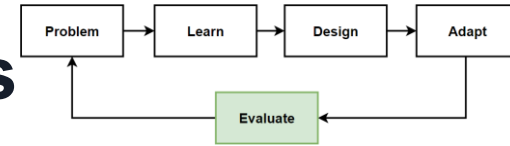
Before Design – What we have in our design toolbox ?

Architectures	Patterns&Principles	Non-FR	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications • HTTP Based RESTful API design • GraphQL API design	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Design: Microservices Architecture with GraphQL APIs



Evaluate: Microservice Architecture with GraphQL APIs

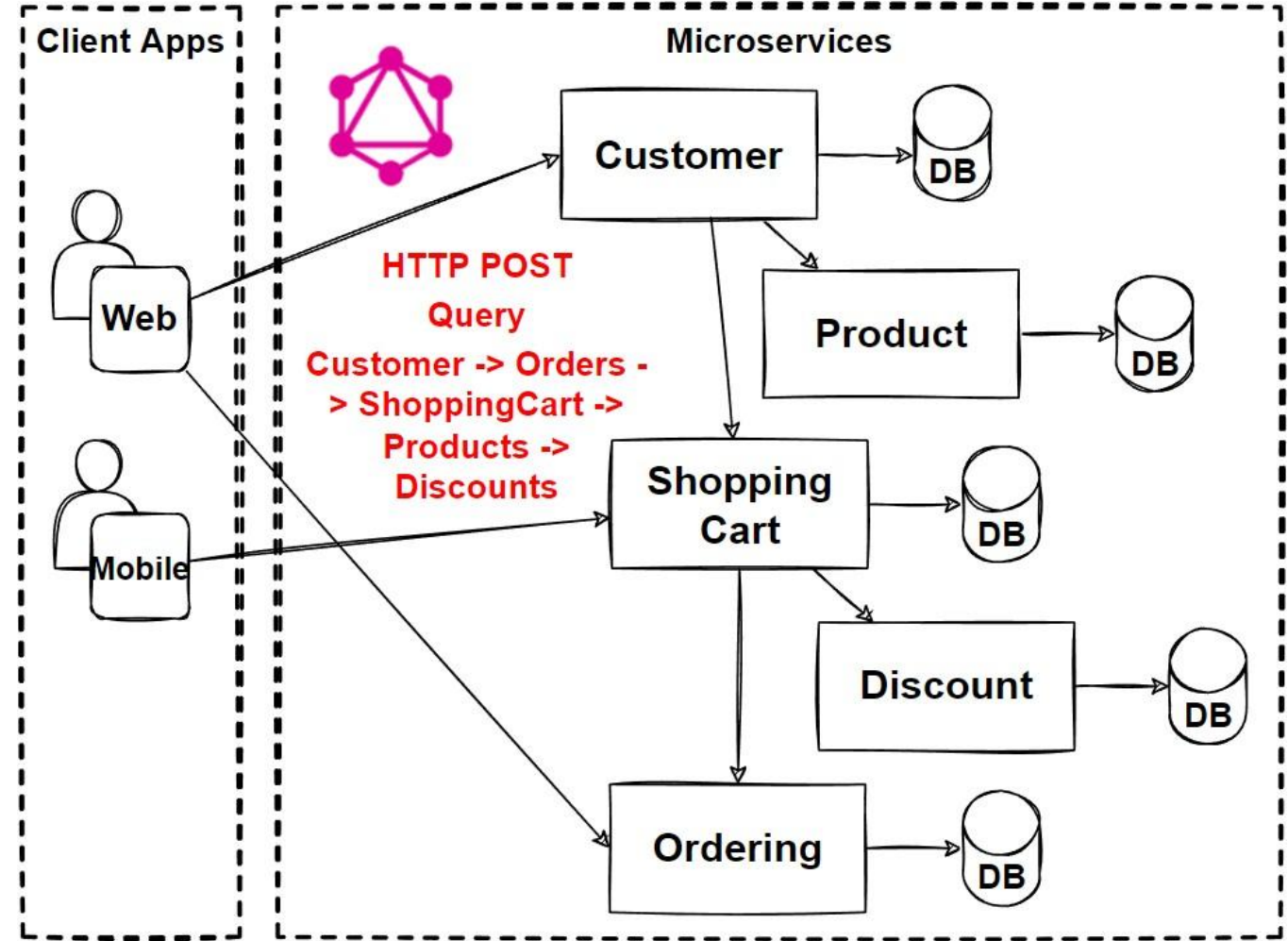


Benefits

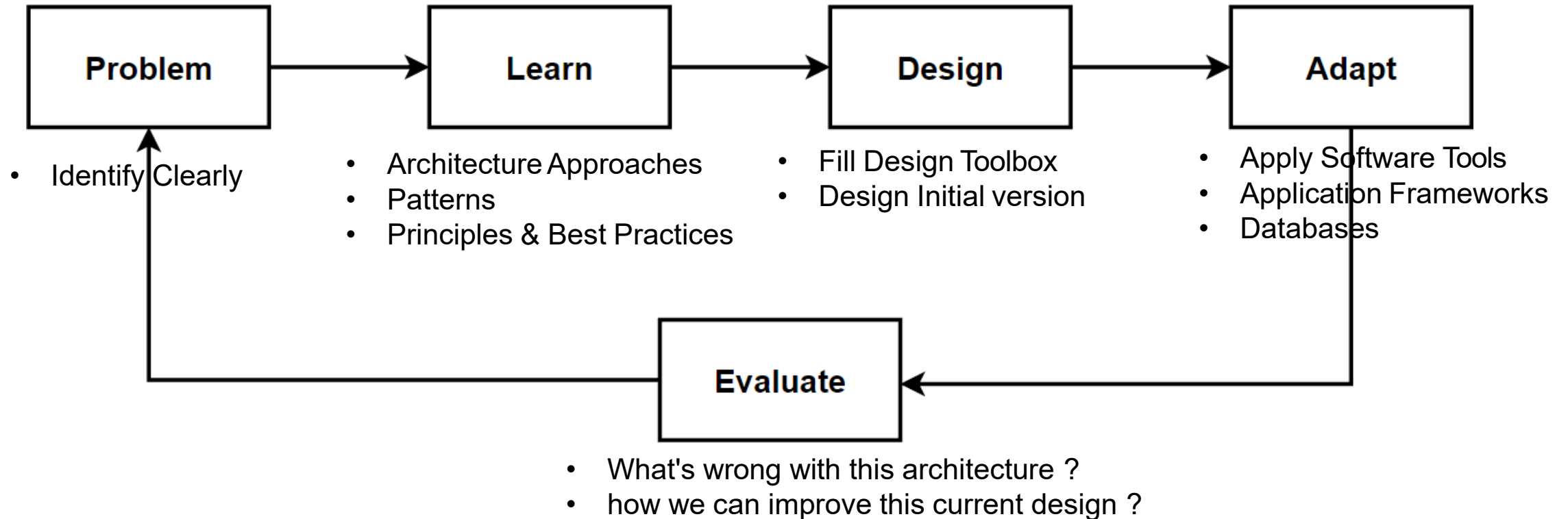
- Solving the over and under-fetching problem
- More flexible with schema and type system
- Quick response time
- Protocol agnostic

Drawbacks

- Increase complexity
- Caching problems



Way of Learning – The Course Flow



Designing Sync APIs for Microservices

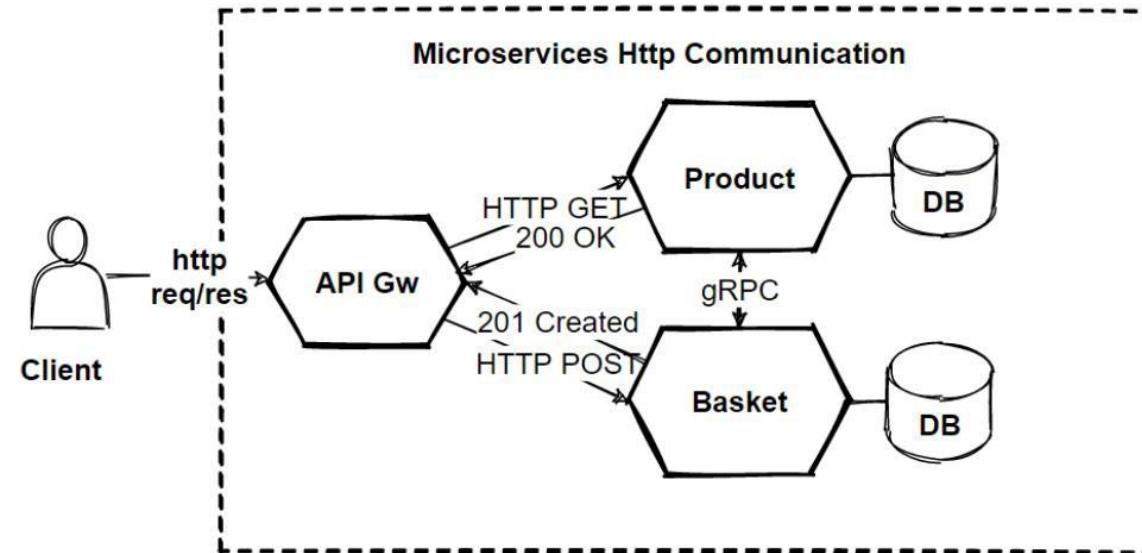
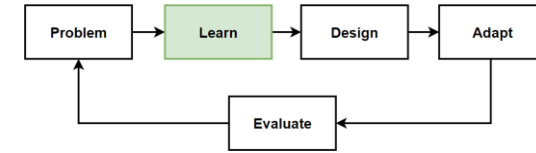
- There are 2 type APIs sync communication in microservices:
 - Public APIs: API calls from the client apps.
 - Backend APIs: inter-service communication between backend services.

- Public APIs**

Use RESTful APIs over HTTP protocol. RESTful APIs use JSON payloads for request-response, that easy to check payloads and easy agreement with clients.

- Backend APIs**

Inter-service communication can result in a lot of network traffic. serialization speed and payload size become more important. Using gRPC is mandatory for increase network performance.



Problem: Inter-service communication makes heavy load on network traffic

Problems

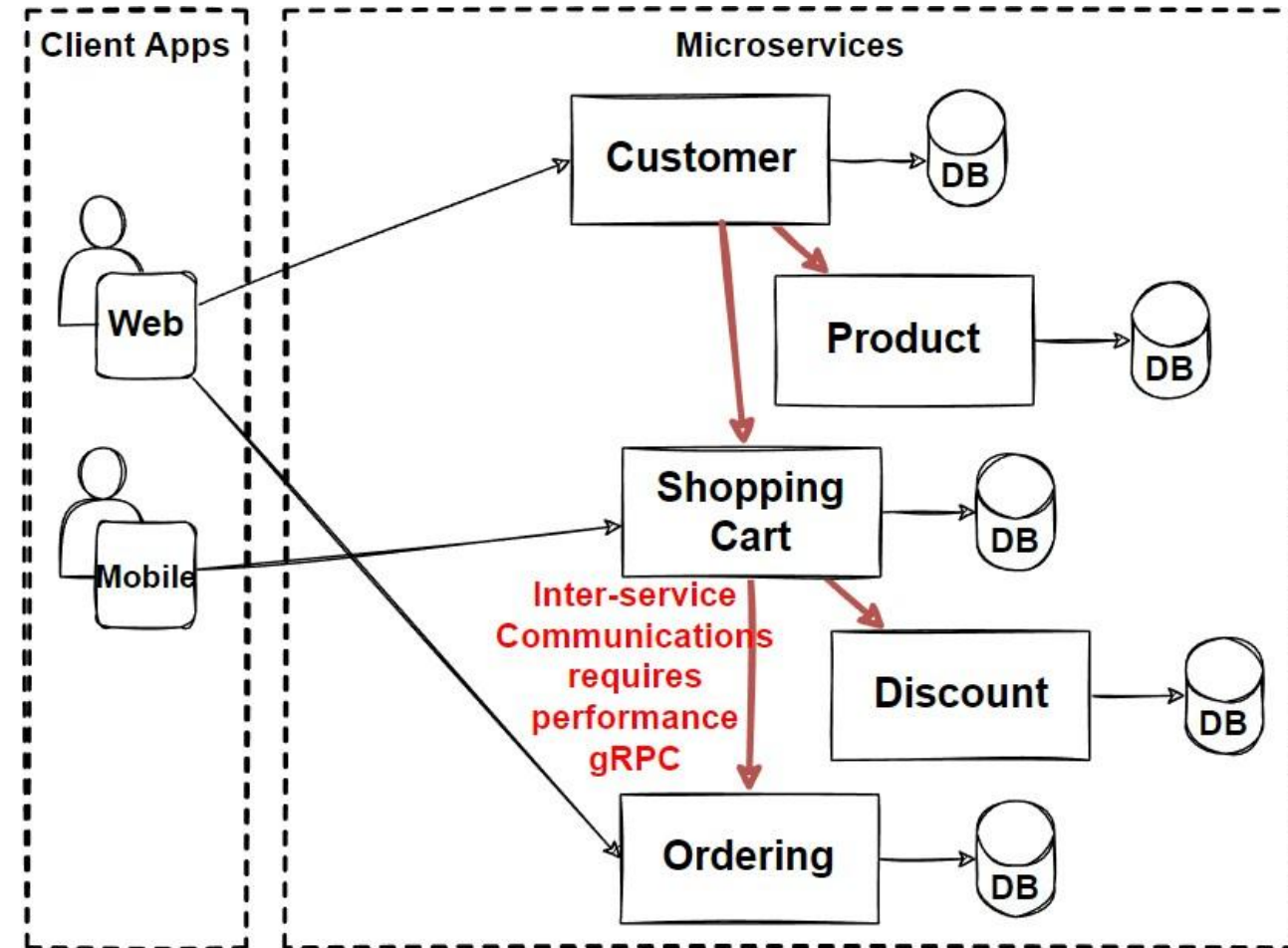
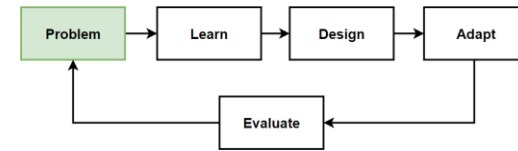
- Network performance issues on inter-service communication
- Backend Communication performance requirements
- Real-time communication requirements
- Streaming requirements

Example Use Case

- Add Item into Shopping Cart that need to calculate with up-to-date discounts

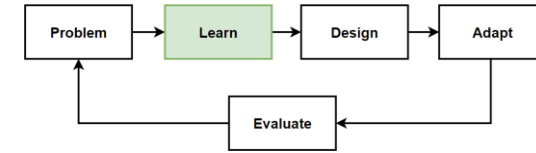
Solutions

- gRPC APIs scalable and fast APIs
- Able to develop with different technologies with RPC framework



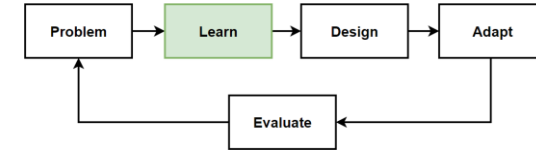
gRPC: High Performance Remote Procedure Calls

- **gRPC** is an open source **remote procedure call (RPC)** system developed at Google.
- gRPC is a framework to **efficiently connect services** and build distributed systems.
- It is **focused on high performance** and uses the **HTTP/2 protocol** to transport binary messages.
- It relies on the **Protocol Buffers** language to define service contracts.
- **Protocol Buffers (Protobuf)**, allow to define the interface to be used in service to service communication regardless of the programming language.
- It generates **cross-platform client** and **server bindings** for many languages.
- Most common **usage** scenarios include connecting services in **microservices style architecture**.



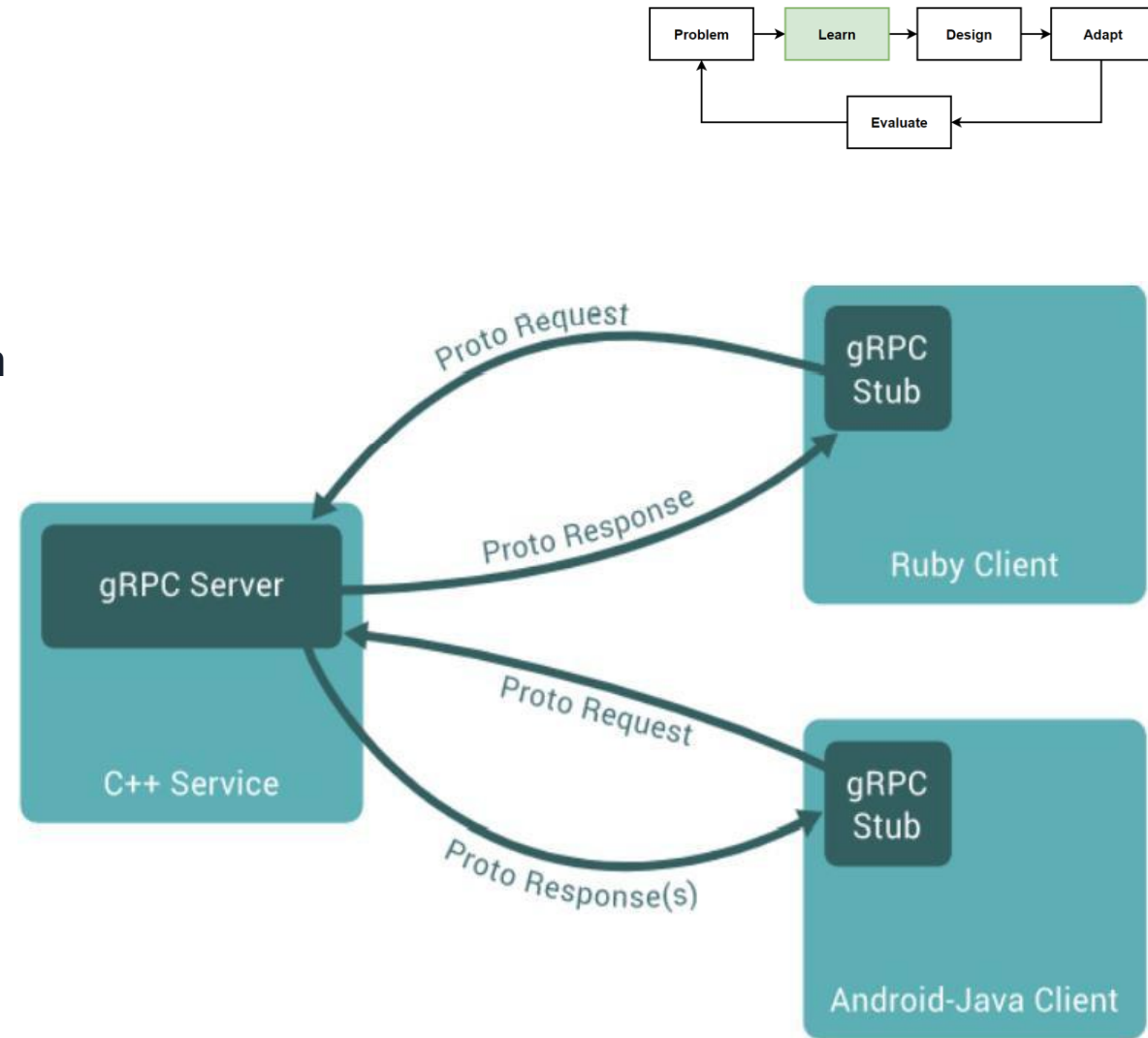
gRPC: High Performance Remote Procedure Calls

- **gRPC framework** allows developers to create services that can communicate with each other **efficiently** and **independently** with their **preferred programming language**.
- Once you **define** a **contract** with **Protobuf**, this contract used by each service to automatically **generate** the **code** that sets up the communication infrastructure.
- This feature simplifies the creation of service interaction and together with **high performance**, makes **gRPC** the **ideal framework** for creating **microservices**.



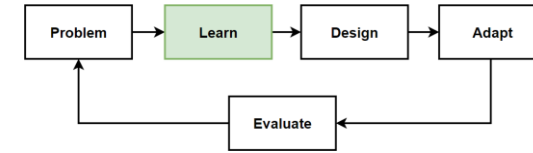
How gRPC works ?

- **RPC** is a form of **Client-Server Communication** that uses a **function call** rather than a usual HTTP call.
- In **GRPC**, client application can **directly call a method** on a server application on a **different machine** like a local object.
- **gRPC** is based on the idea of **defining a service** and **methods**, and these can be called remotely with their parameters and return types.
- On the **server side**, the server implements this interface and runs a gRPC server to handle client calls.
- On the **client side**, the client has a stub that provides the same methods as the server.
- **gRPC clients** and **servers** can work and talk to each other in a different of environments.



Main Advantages of gRPC

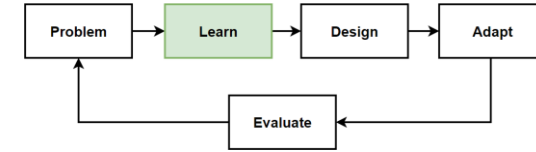
- Using **HTTP/2** provides **30-40% more performance**.
- **gRPC** uses **binary serialization**, provides **higher performance** and **less bandwidth** usage than **json** with binary serialization.
- Supporting a widely using **multi-languages** and **platform** supports.
- **Open Source** and the powerful community behind it.
- Supports **Bi-directional Streaming** operations.
- Support **SSL/TLS** usage.
- Supports many **Authentication methods**.



gRPC

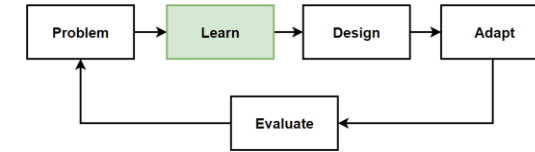
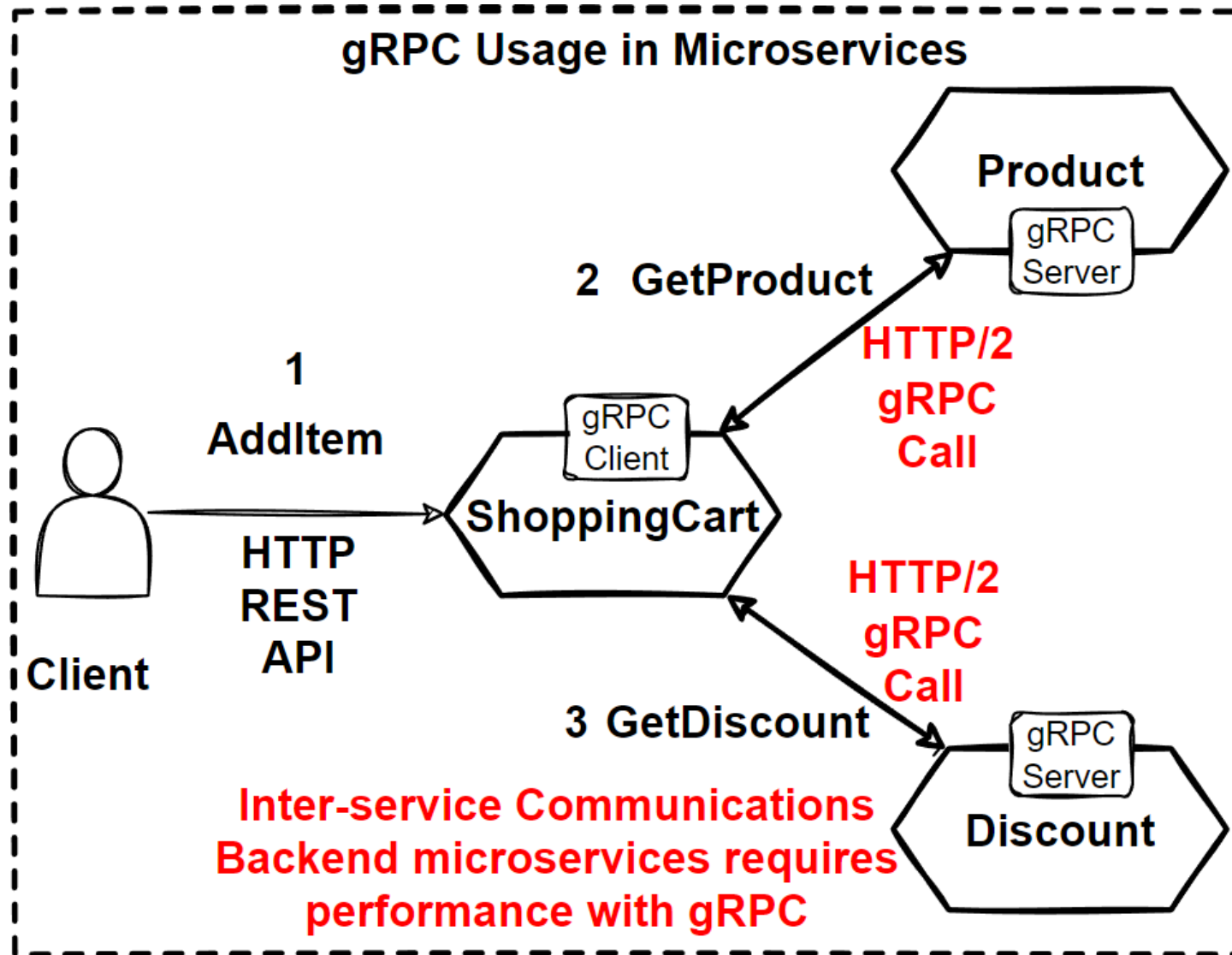
When to use gRPC ? Use Cases of gRPC

- **gRPC** is primarily used with **backend services**.
- **Synchronous backend microservice-to-microservice communication** where an immediate sync response is required to continue processing.
- **Polyglot environments** that need to **support mixed programming platforms**.
- **Low latency** and **high throughput communication** where performance is critical.
- **Point-to-point real-time communication** that gRPC can push messages in real time and requires to support for bi-directional streaming.
- **Network constrained environments** which requires **less bandwidth usage** cases, binary gRPC messages are always smaller than text-based JSON message.



gRPC

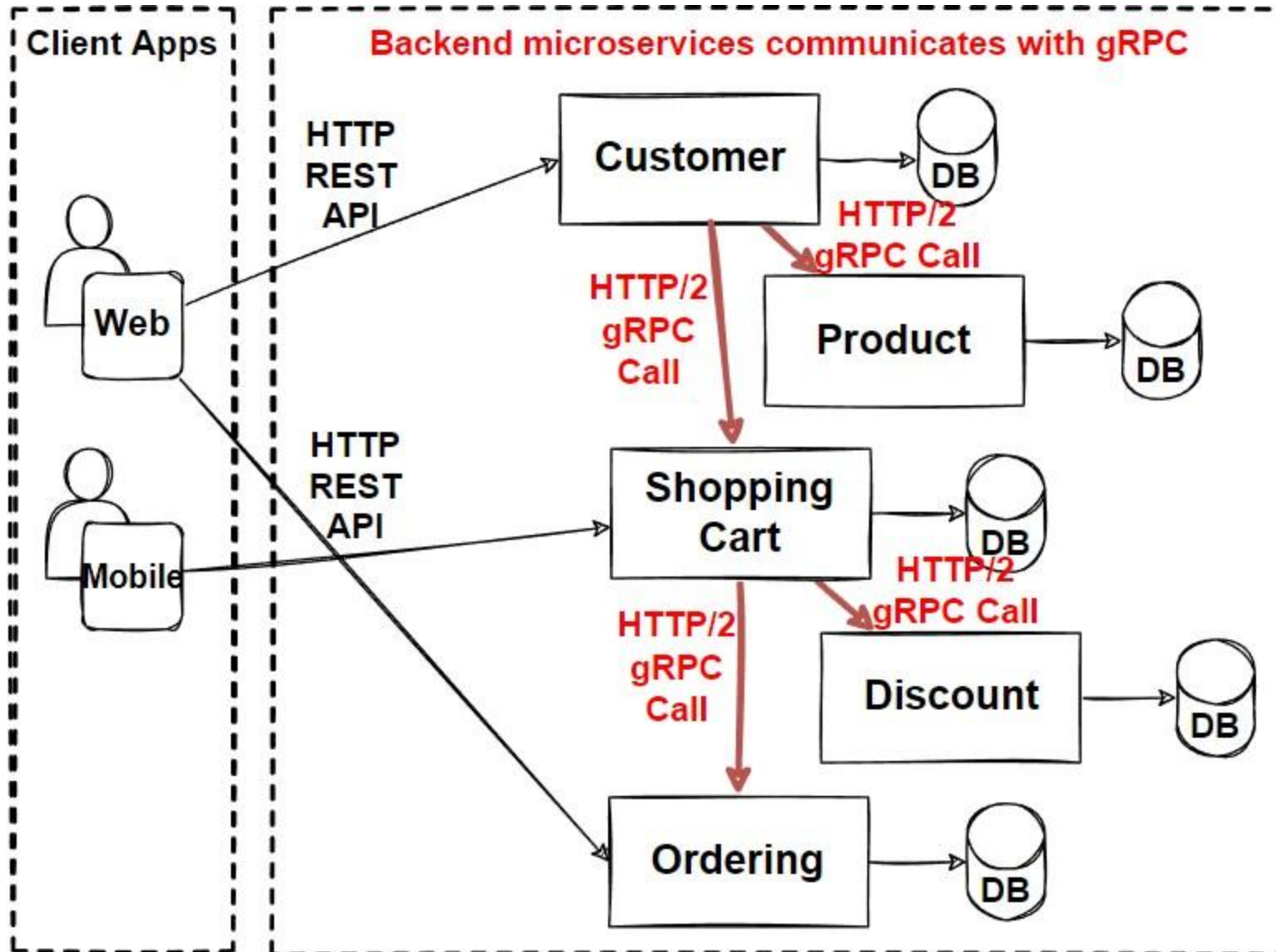
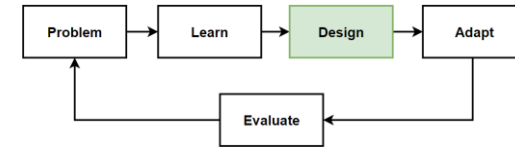
gRPC Usage in Microservices Communication



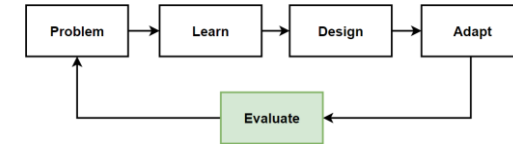
Before Design – What we have in our design toolbox ?

Architectures	Patterns&Principles	Non-FR	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications • HTTP Based RESTful API design • GraphQL API design • gRPC API Design	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Design: Microservices Architecture with gRPC APIs



Evaluate: Microservice Architecture with gRPC APIs

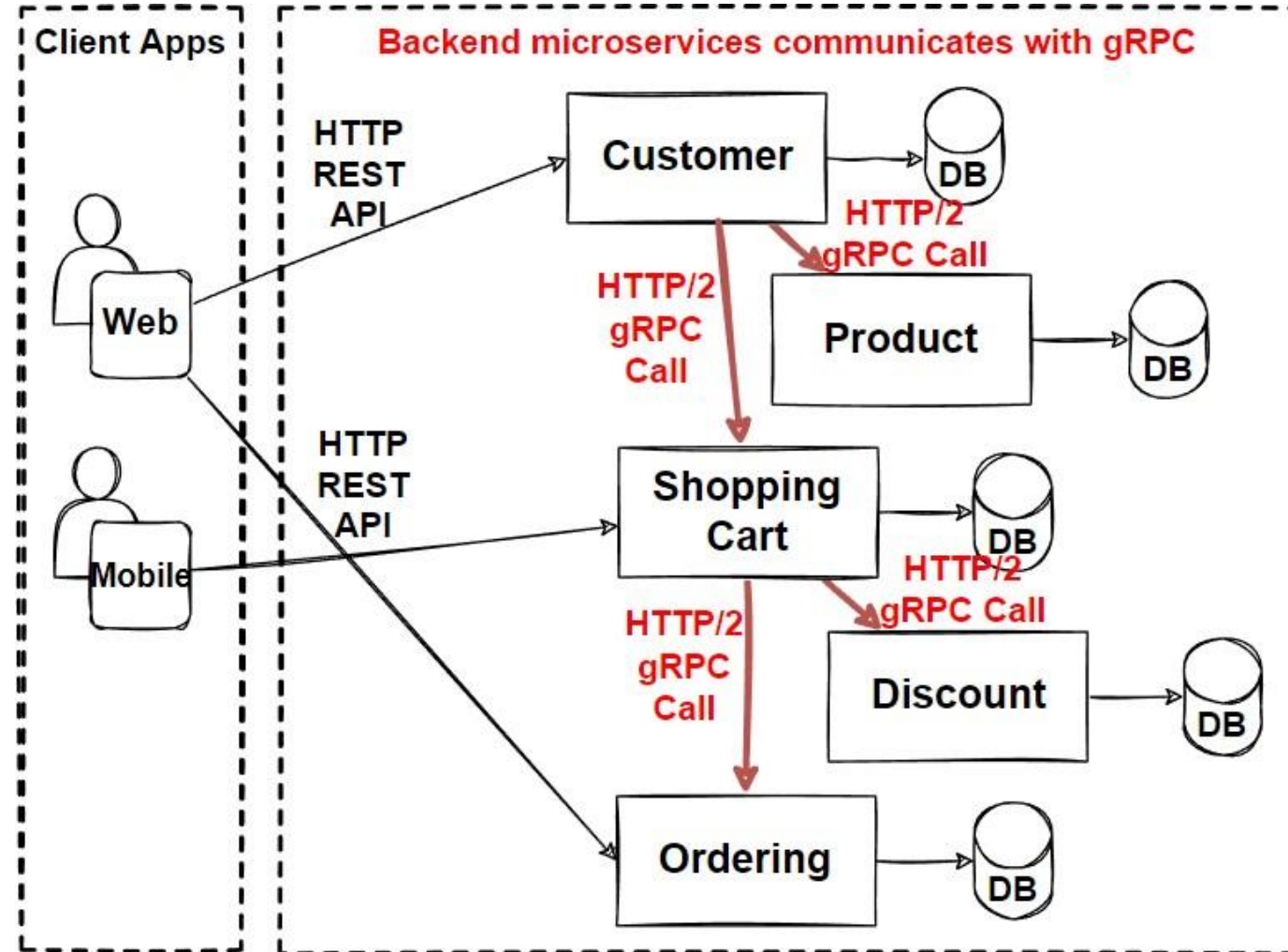


Benefits

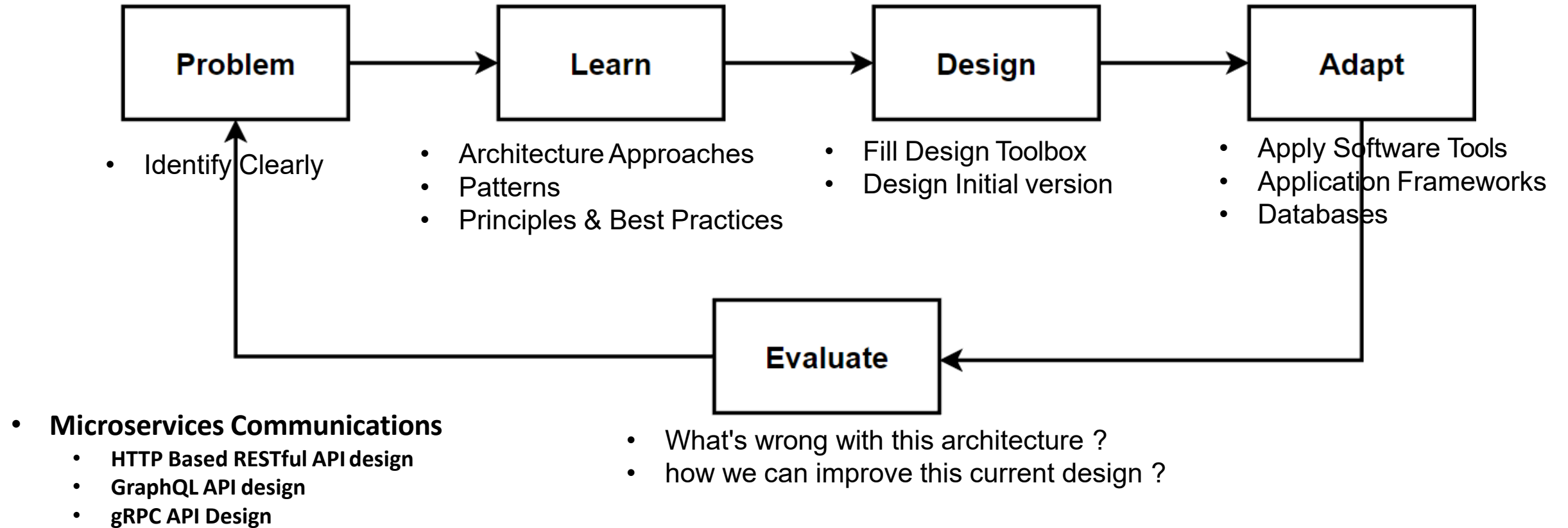
- High-performance communication
- Less bandwidth usage
- Multi-language/platform supports
- HTTP/2 and SSL/TLS usage
- Supports Bi-directional Streaming operations

Drawbacks

- Need additional library to generate codes that increases complexity
- Can't read incoming-outgoing payloads that is not good for client to service communications



Way of Learning – The Course Flow



Problem: Chat with Support Agent

Problems

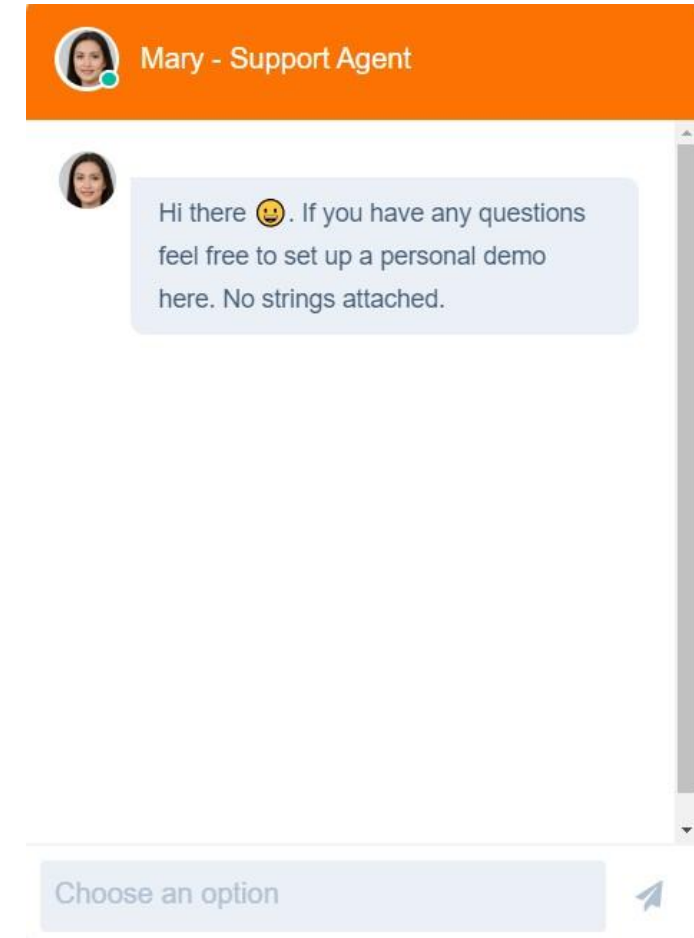
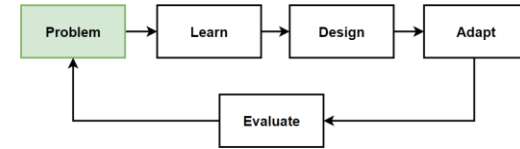
- Business teams request to answer Customer queries by chatting with Support Agents
- Real-time communication requirements
- Sending/receiving messages in Chat window

Example Use Case

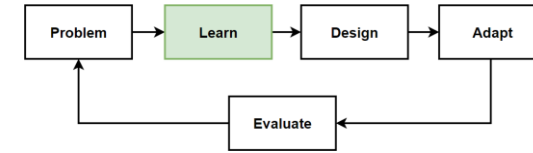
- E-commerce Online Agent help customer preferences as per product features on website

Solutions

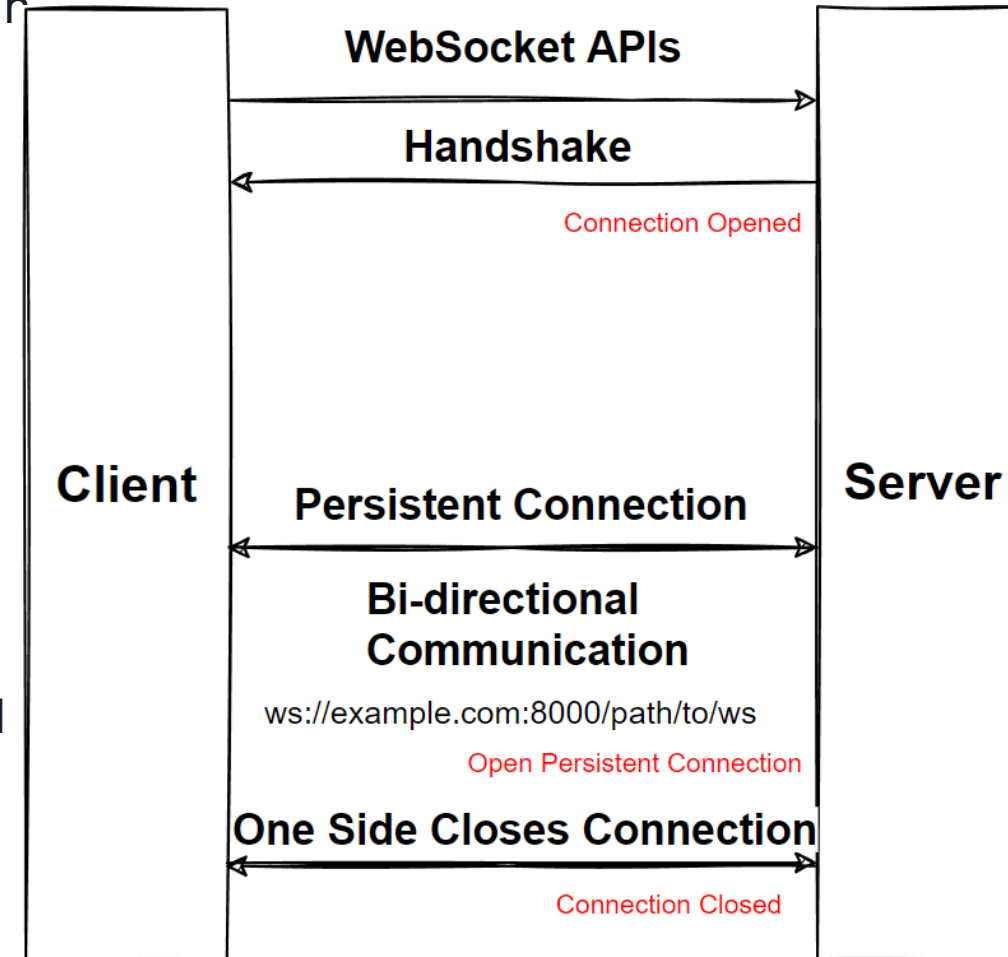
- WebSocket APIs: Build real-time two-way communication applications



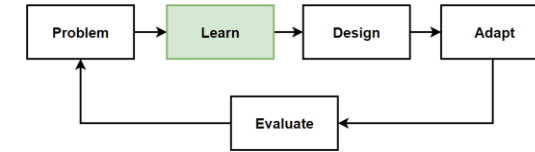
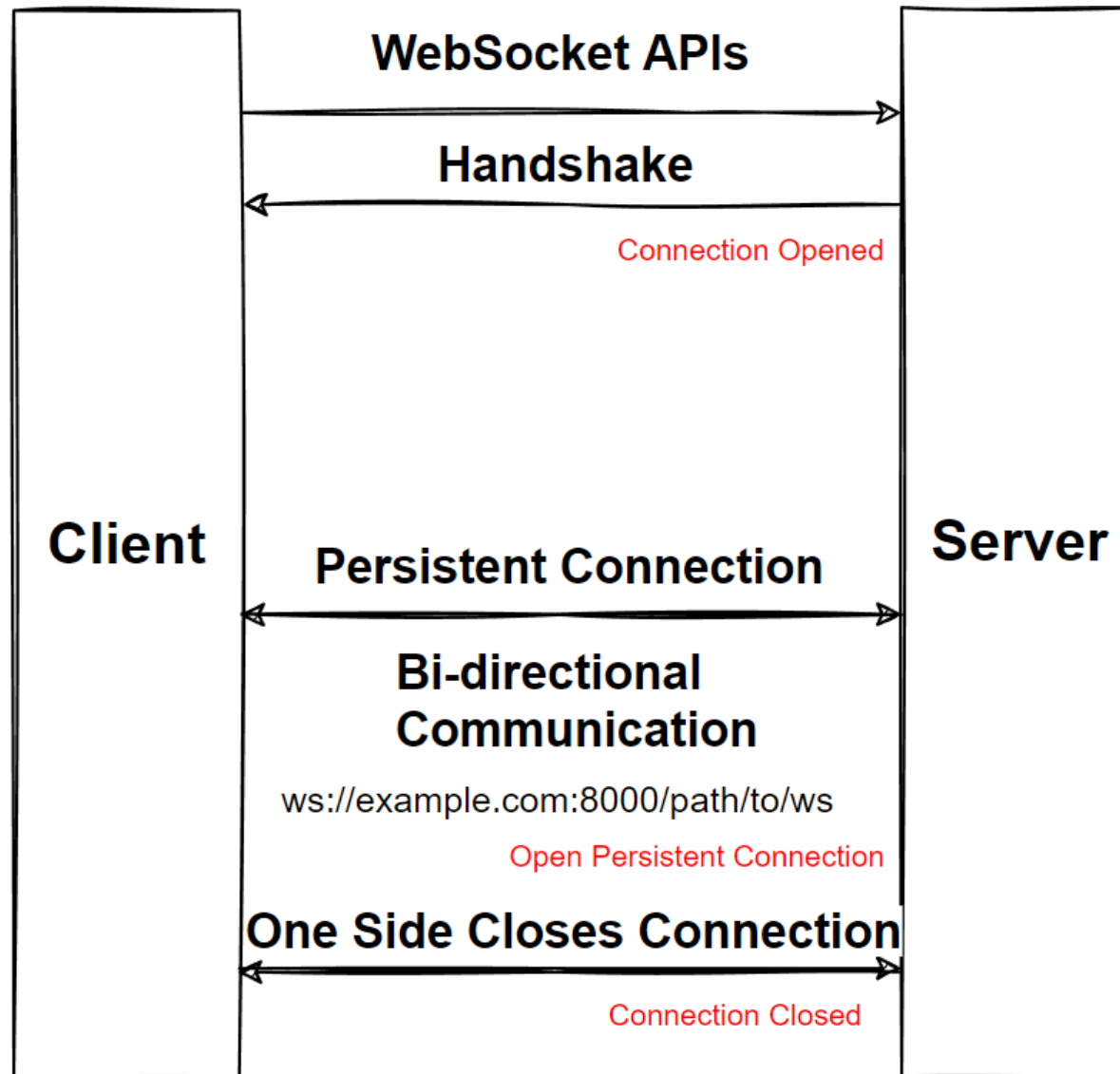
WebSocket API: Real-time two-way communications



- **WebSocket API** is an advanced technology that user can establish an **interactive two-way communication session** between **browser** and a **server**.
- **Send a message** to a **server** and **receive event-based responses** without having to **poll** the server for a response.
- **WebSockets** is a good way to handle high-scale data transfers between server-client.
- **WebSocket** is **bidirectional**, it starts with **ws://** or **wss://**
- Unlike HTTP, WebSocket provides a **bidirectional protocol** used in the **same client-server communication** cases.
- **WebSocket** is a **stateful protocol**, connection between client and server stays alive until terminated by one of the parties.
- After closing the connection by either client or server, the **connection** is **terminated** from **both ends**.



WebSocket API: Real-time two-way communications



When to use WebSocket API

- **Developing Real-time Web application**

The most usage area of WebSocket is in real-time application development where it is required to continuous display of data at the client end. In WebSocket, data is constantly pushed/forwarded to the same connection that is already open.

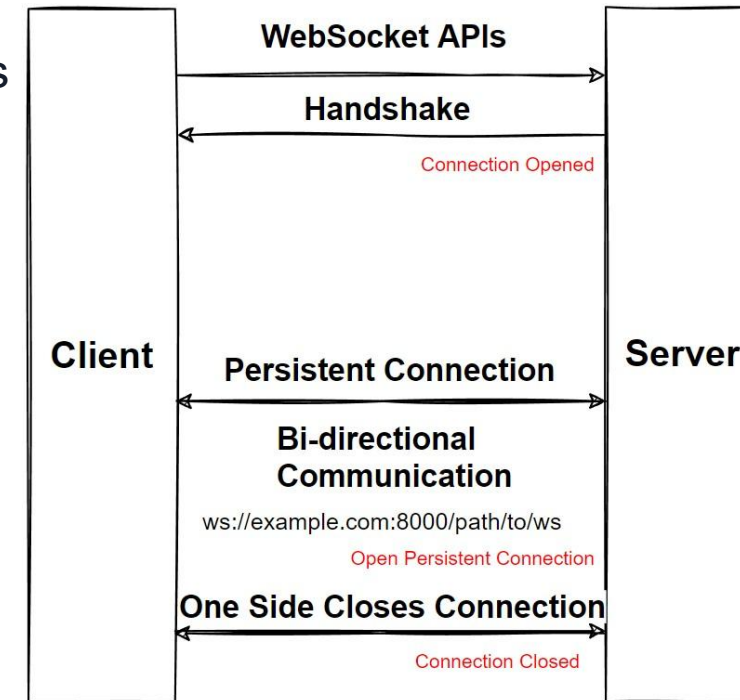
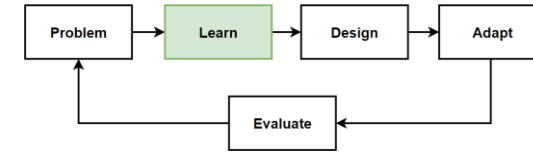
- Real-life example of WebSocket is stock trading applications. WebSocket helps to data handling that is drives by the deployed backend server to the client.

- **Developing Game application**

Gaming application requires to changes on the UI with WebSockets without updating the page and refreshing UI. WebSocket accomplish this goal without disturbing the UI of the gaming app.

- **Developing Chat application**

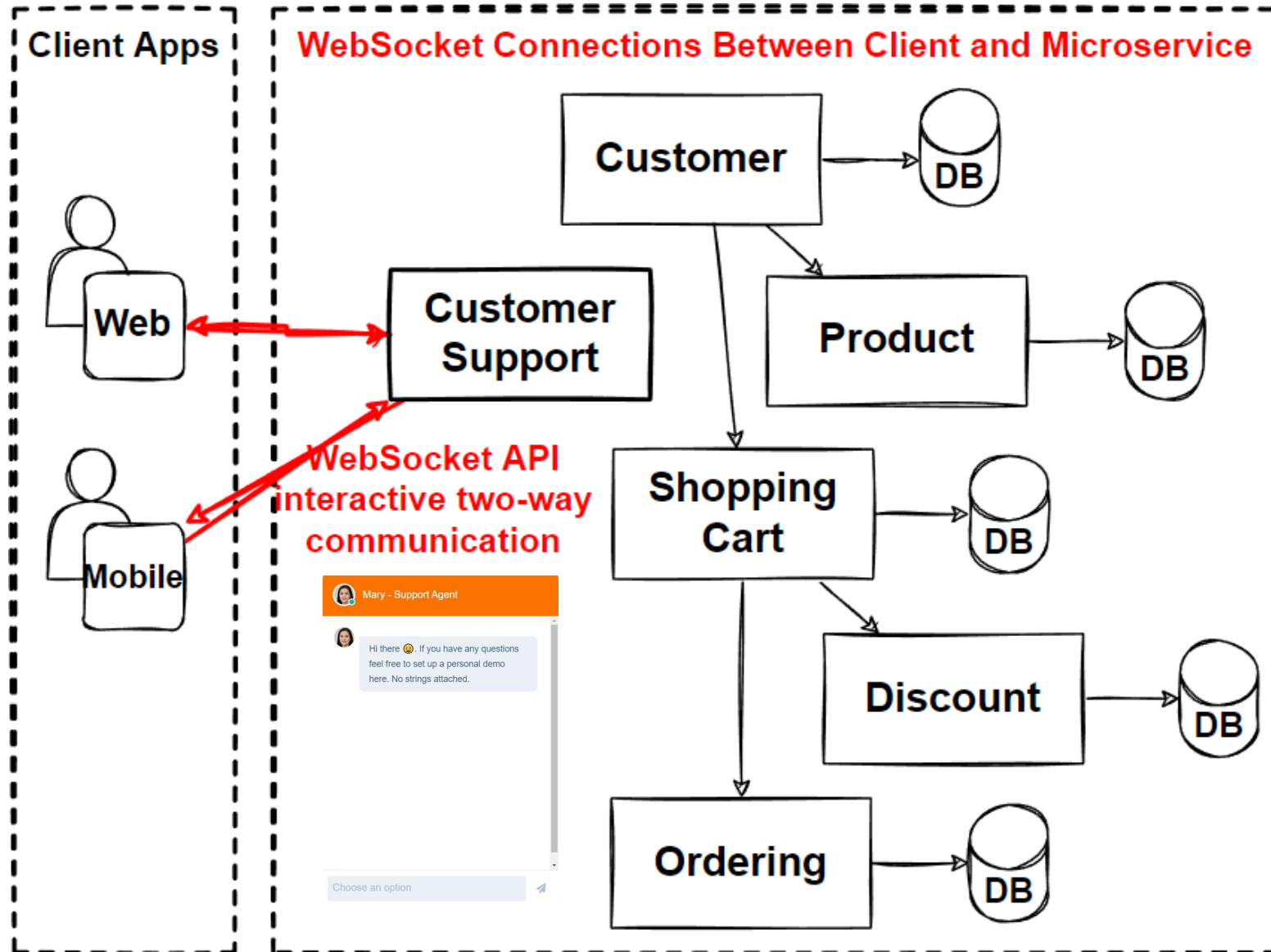
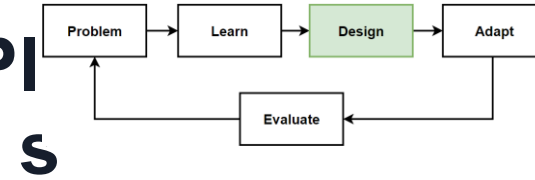
Chat application uses WebSocket to use fixed connection when exchange the message between subscriber, broadcasting channels. WebSocket connection is used for sending/receiving messages, communication becomes easy and quick.



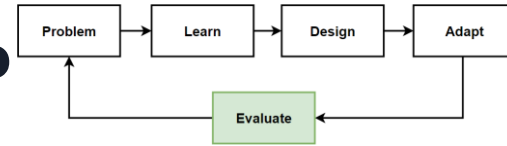
Before Design – What we have in our design toolbox ?

Architectures	Patterns&Principles	Non-FR	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications • HTTP Based RESTful API design • GraphQL API design • gRPC API Design • WebSocket API	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Design:Microservices Architecture with WebSocket API



Evaluate: Microservice Architecture with WebSocket AP

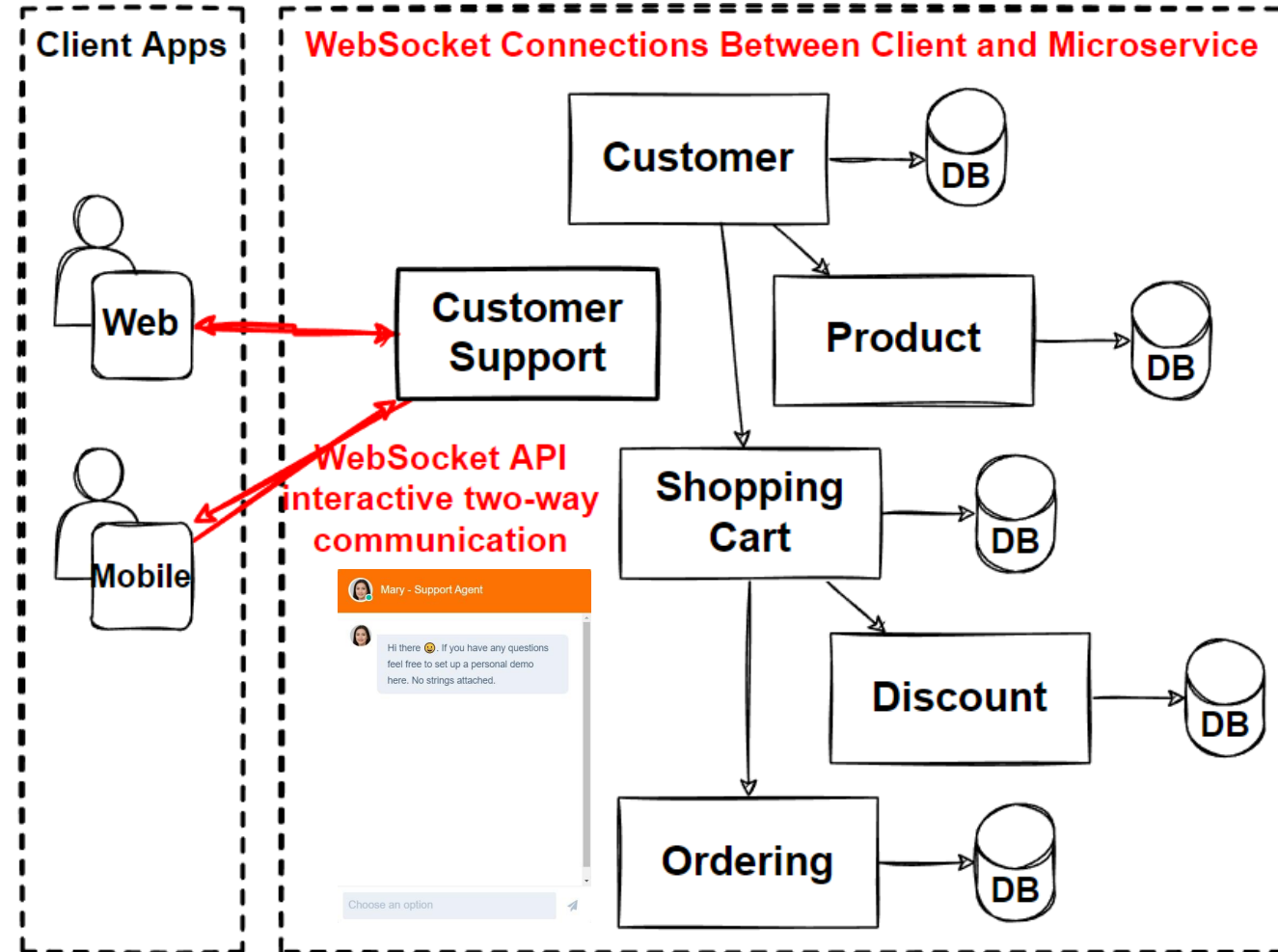


Benefits

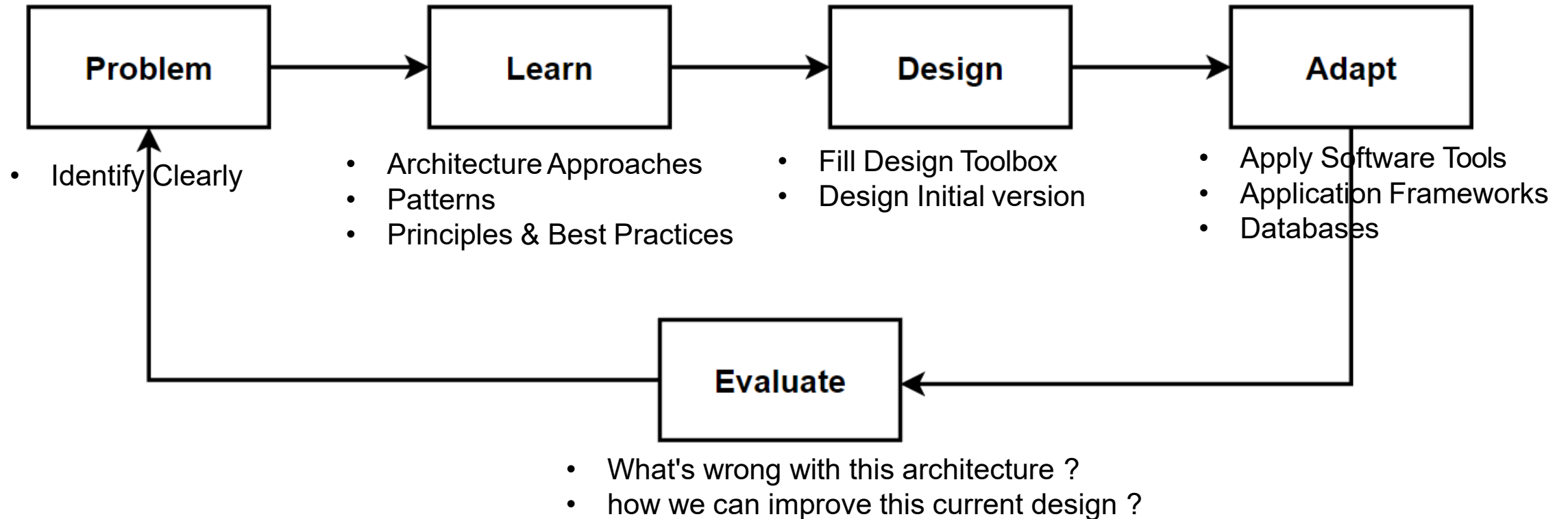
- Full-duplex communication
- Reduces unnecessary network traffic
- Real-time communication
- Persistent, Bidirectional Communication

Drawbacks

- A fully HTML5-compliant web browser is required.
- Not provide intermediary/edge caching.



Way of Learning – The Course Flow

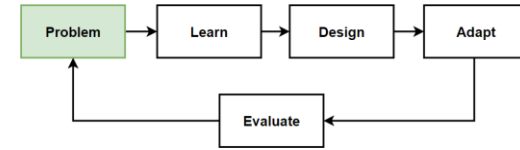


```
graph LR; Problem[Problem] --> Learn[Learn]; Learn --> Design[Design]; Design --> Adapt[Adapt]; Adapt --> Evaluate[Evaluate]; Evaluate --> Problem;
```

-
- The diagram illustrates a synchronous communication flow between a Client and a series of microservices. The Client, represented by a person icon, sends a request to the **Basket** service. The services are connected in a chain: **Basket** ↔ **Product** ↔ **Pricing**. Below this chain, there is an **Order** service, which is connected to an ellipsis (...) indicating other services. A vertical double-headed arrow between the **Pricing** and **Order** services is labeled **Http / gRPC Request/Responses**, indicating the communication protocol. The entire system is enclosed in a dashed box.

- How client applications can manage all these different protocols and communications inside of their applications ?

Problem: Direct Client-to-Service Communication



Problems

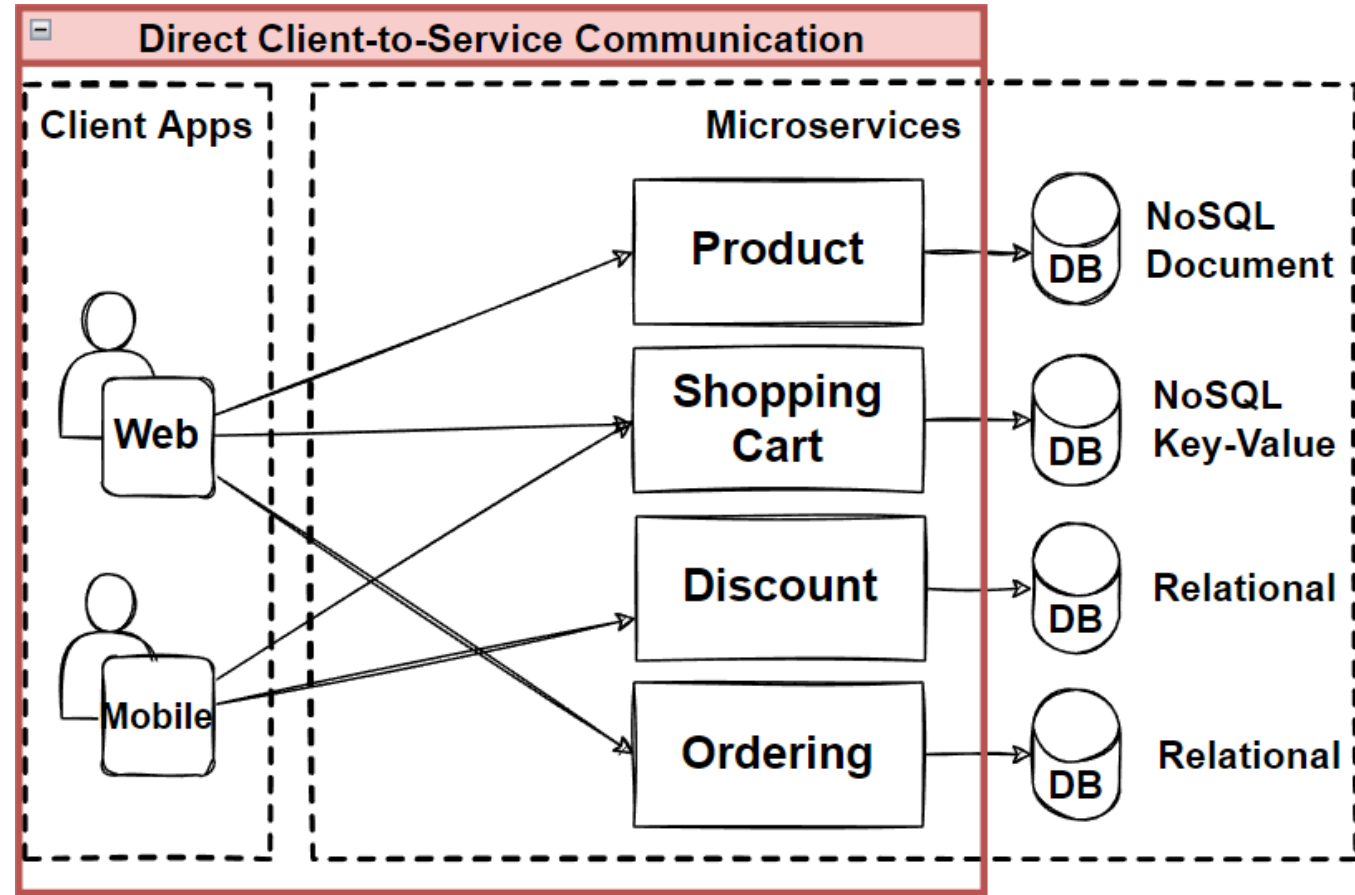
- Direct Client-to-Service Communication
- Cause to chatty calls from client to service
- Hard to manage invocations from client app with all different protocols (HTTP, GraphQL, gRPC, WebSocket)

Solutions

- Well-defined API Design
- **Microservices Communication Patterns**

Steps

- Microservice Communications and Types
- Well-defined RESTful API Design for services
- **Microservices Communication Patterns (API Gateway, BFF, Publish/Subscribe..)**



**CÁM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**